CSE 4102: Computer Graphics and Image Processing Laboratory

PROJECT REPORT

ON

SmartGarage: Automatic Vehicle Inspection and Service System Using OpenGL

Submitted To	Submitted By
1. Md. Tajmilur Rahman Lecturer **2. Md. Mubtashim Abrar Nihal** Lecturer	**Afifa Sultana Offa** Roll: **2107087** Year: Fourth Semester: First

Submission Date: October 7, 2026

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

Abstract

SmartGarage is an OpenGL 3.3 (core profile) application in which a Cybertruck-style vehicle waits outside a smart service garage, the roller door opens, the vehicle drives in and climbs a ramp onto a scissor lift, the door closes, the lift rises, and a five-joint robotic arm inspects the vehicle with a moving spotlight, cleans and polishes it, tightens the wheel fasteners, and finally the lift lowers, the door opens and the vehicle reverses out. The whole sequence is an 18-state machine lasting 197.7 s. Every object is built in code from a reusable cube, two cylinders and a sphere, except the vehicle shell, which is a custom triangle mesh. All 21 items of the original proposal are implemented — hierarchical modelling, transformations, directional, point, spot and moving lights, ambient–diffuse–specular lighting, **Gouraud** and **Phong** shading, motion and interaction — and are extended with Cook–Torrance PBR, shadow mapping, textures, HDR post-processing collision handling, and a physical inspection monitor whose picture is produced by genuine *ray tracing* with shadow rays (the rest of the garage stays rasterised). The report gives the real construction sequence of the vehicle, lift and robot with the calculations behind each step, an equation and figure for every other object, and for every part its source file, variables and the effect of a change.

Keywords: OpenGL, GLSL, hierarchical modelling, Phong reflection, Gouraud shading, Phong shading, animation, robotic arm.

Contents

1 Introduction **3**

2 Proposal-to-Final Implementation Mapping **3**

3 Object Modelling and Construction **4**

4 Transformations, Camera and Projection **14**

5 Lighting, Shading, Materials and Textures **15**

6 Ray-Traced Underbody Inspection Monitor **19**

7 Motion, Animation, Interaction and Collision **23**

8 Automatic Service Workflow **24**

9 Requirement Fulfilment and Final Verification **24**

10 Conclusion **24**

1 Introduction

Background. Real-time graphics renders triangles through vertex processing, rasterisation and fragment shading [1, 2]. SmartGarage uses one scenario — an automated service bay — to demonstrate the course topics: geometry, transformation, lighting, shading, motion and interaction.

Objectives. (1) generate primitives procedurally and reuse them through model matrices; (2) build the garage, vehicle, lift and robot hierarchically; (3) implement the Phong lighting model with Gouraud and Phong shading and four light types; (4) animate the full inspection sequence; (5) let the user control the camera, door, lift, robot, lights and shading. The six mandatory course requirements are *lighting, transformation, Gouraud shading, Phong shading, motion/animation* and *interaction*; ray tracing is an optional bonus and is implemented for one dedicated view, the underbody inspection monitor of Section 6; the rest of the garage is rasterised.

Final concept. Interior $28 \times 26 \times 7\,\mathrm{m}$ ($x \in [-14, 14]$, $z \in [-13, 13]$, 1 unit = 1 m); entrance $13 \times 4.9\,\mathrm{m}$ at $z = 13.2$; lift centred at $z = -2$; robot bolted at $(-4.6, 0, -2.2)$; tool table at $(-7.9, 0, -1.85)$. The vehicle starts on the apron at $z = 16.5$. The source is organised as `Math3D.h` (maths), `Camera.h`, `Primitives.cpp` (meshes), `SceneObjects.cpp` (all objects), `Shader.cpp` (GLSL) and `Main.cpp` (state machine, input, collision, lights, render passes); the function names are given with every object below.

2 Proposal-to-Final Implementation Mapping

Every item of the original proposal is checked against the final source in Table 1; the **Phong shading mode**, the **headlight point lights** and the **vehicle exit** were completed in final preparation. Figures 5, 13, 14 and 30 show the storyline rendered by the program.

Table 1: Proposal items versus final implementation. Result: **21 of 21 implemented** (14 fully, 6 with improvement, 1 implemented differently but equivalently).

Proposed feature	Proposal expectation	Final implementation (how)	File / function	Evidence	Status
Scene objects					
Automobile	body, wheels, windows, headlights from primitives; moves in, wheels rotate	custom triangle-mesh shell (13 batches) + cylinder wheels; wheel angle $\theta = s/r$ from distance travelled	`drawCar`, `drawWheel`, `updateCarDrive`	3	Implemented with improvement
Garage entrance door	sliding door, opens / closes	5-panel roller shutter sliding in guide rails, winding onto a drum	`drawGarageDoor`	13	Implemented differently but equivalently
Hydraulic lift	platform moves up/down by transformation	double-scissor lift; angle solved from height, platform top $= 1.08 + h$	`scissorAngle`, `drawHydraulicLift`	6	Implemented with improvement
Robotic inspection arm	primitives, scanner at end; joints rotate	5-DOF arm (yaw, shoulder, elbow, wrist pitch/roll) + 4 exchangeable tools	`drawRobotArm`, `runToolChange`	8	Implemented with improvement
Inspection light fixture	spotlight on arm; ceiling lights as instances	spotlight from arm FK; 8 instances of one ceiling fixture	`uploadSceneUniforms`, `drawCeilingFixture`	21	Fully implemented
Garage environment	floor, walls, ceiling from surfaces	scaled cubes + beams, columns, props, textured floor	`drawGarage`, `drawExterior`	16	Implemented with improvement
Lighting design					
Directional (ceiling)	fixed direction, overall light	directional key $\hat{\mathbf{d}} = (0.166, -0.922, -0.350)$ with shadow map; ceiling has 8 point lights	`buildLightSpaceMatrix`	21a	Fully implemented
Point light in headlights	inside each headlight, falloff	two point lights inside the front light channel, carried by the car matrix; $A = 1/(1{+}0.075d{+}0.018d^2)$	`headlightLocalPosition`, `uploadSceneUniforms`	20c,d,f	Fully implemented
Spotlight (scanner)	mounted on arm, moves	position/direction from the wrist and tool tip every frame	`robotSafetyPoints`, Shader `phongReflection`	20a,b,e	Fully implemented
Ambient light	base illumination	$k_a L_a = 0.038$ in the Phong model; image-based ambient in PBR	`phongReflection`	20	Fully implemented
Animation flow					
Step 1 arrival	door opens, vehicle enters, wheels rotate, stops on lift	DOOR_OPEN_IN, DRIVE_IN; ramp climb with pitch	`updateServiceAutomation`	14	Implemented with improvement
Step 2 positioning	door closes, aligns	DOOR_CLOSE; vehicle stops at $z = -2$ on the turntable	same	30	Fully implemented
Step 3 inspection	lift raises, arm scans, spotlight follows	LIFT_UP; SCAN of 6 regions, turntable shows far side	same	5	Implemented with improvement
Step 4 lighting demo	headlights on, point lights light nearby surfaces, Gouraud vs Phong	headlights on by default (key '); `F5` cycles PBR / Gouraud / Phong	`handleDiscreteKey`	20	Fully implemented
Step 5 completion	lift lowers, door opens, vehicle exits	LIFT_DOWN, DOOR_OPEN, DRIVE_OUT (vehicle reverses onto apron)	same	30	Fully implemented
Interactive features					

Proposed feature	Proposal expectation	Final implementation (how)	File / function	Evidence	Status
Move camera	around garage	`W A S D`, `Space/Ctrl`, mouse, scroll	`processMovement`, `Camera`	–	Fully implemented
Headlights ON/OFF	toggle	key ’ (light bars + point lights)	`handleDiscreteKey`	20c,f	Fully implemented
Control spotlight movement	move the lamp	robot keys `T/G U/J I/K` move the scanner and its light	`processMovement`	9	Fully implemented
Rotate inspection light	manual rotation	`Y/H` wrist pitch, ← / → wrist roll	same	–	Fully implemented
Switch Gouraud / Phong	shading modes	`F5`: PBR → Gouraud → Phong	Shader `uShadingMode`	20	Fully implemented
Lighting on materials	observe differences	metal / plastic / glass / rubber materials, shininess from roughness	`inferMaterialType`	22	Fully implemented

Enhancements beyond the proposal: a ray-traced underbody inspection monitor with shadow rays (Section 6); Cook–Torrance PBR (anisotropic stainless, clear coat, image-based lighting); directional and spot shadow maps; triplanar texture array; HDR, MSAA, bloom, ACES; perspective/orthographic switch; cleaning, polishing and wheel service with visible tool exchange; turntable; collision prevention; Bézier hoses; teaching mode showing construction stages.

3 Object Modelling and Construction

3.1 Primitive generation (how the vertices are found)

Cube (`kCubeVertices`): $[-0.5, 0.5]^3$, 24 vertices (6 faces × 4, so each face keeps its own normal), 36 indices (2 triangles per face, e.g. back face $(0, 1, 2), (2, 3, 0)$, counter-clockwise from outside).

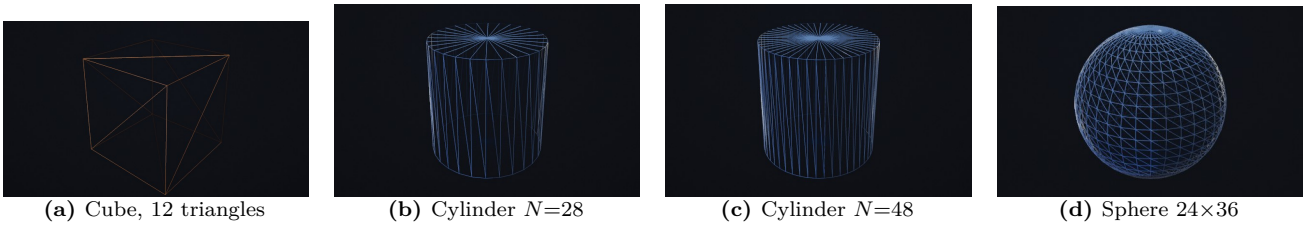
Cylinder (`buildCylinder`, $r = 0.5$, height 1, axis y): for boundary $i = 0..N$,

$$\theta_i = \frac{2\pi i}{N}, \qquad x_i = r\cos\theta_i, \qquad z_i = r\sin\theta_i, \qquad \mathbf{n}_i = (\cos\theta_i, 0, \sin\theta_i). \tag{1}$$

Example (project mesh $N = 28$, $i = 7$): $\theta_7 = 2\pi \cdot 7/28 = \pi/2 = 90°$, so $x_7 = 0.5\cos 90° = 0$, $z_7 = 0.5\sin 90° = 0.5$: vertices $(0, \pm 0.5, 0.5)$, normal $(0, 0, 1)$. For $i = 1$: $\theta_1 = 12.857°$, $(x, z) = (0.4875, 0.1113)$. Side quad i uses indices $(2i, 2i{+}2, 2i{+}3)$ and $(2i{+}3, 2i{+}1, 2i)$; caps are fans around a centre vertex. Totals $V = 4N + 6$, $I = 12N$: 118/336 for $N = 28$ and 198/576 for the smooth $N = 48$ mesh (`drawCylS`). **UV sphere** (`buildSphere`, 24 stacks, 36 sectors, $r = 0.5$):

$$\phi = \pi\tfrac{y}{24},\ \theta = 2\pi\tfrac{x}{36}, \qquad \mathbf{p} = r(\sin\phi\cos\theta,\ \cos\phi,\ \sin\phi\sin\theta), \qquad \mathbf{n} = \mathbf{p}/r. \tag{2}$$

Example: stack 6, sector 9 ⇒ $\phi = 45°, \theta = 90°$ ⇒ $\mathbf{p} = (0, 0.3536, 0.3536)$. Each grid cell (a, b, c, d) gives triangles $(a, b, c), (c, b, d)$: 925 vertices, 1728 triangles. All meshes share one layout (position + normal, 24-byte stride) in a VAO/VBO/EBO created once in `Primitives::init()`; colour is a uniform, so one cube becomes every wall, panel and bolt.



(a) Cube, 12 triangles **(b)** Cylinder N=28 **(c)** Cylinder N=48 **(d)** Sphere 24×36

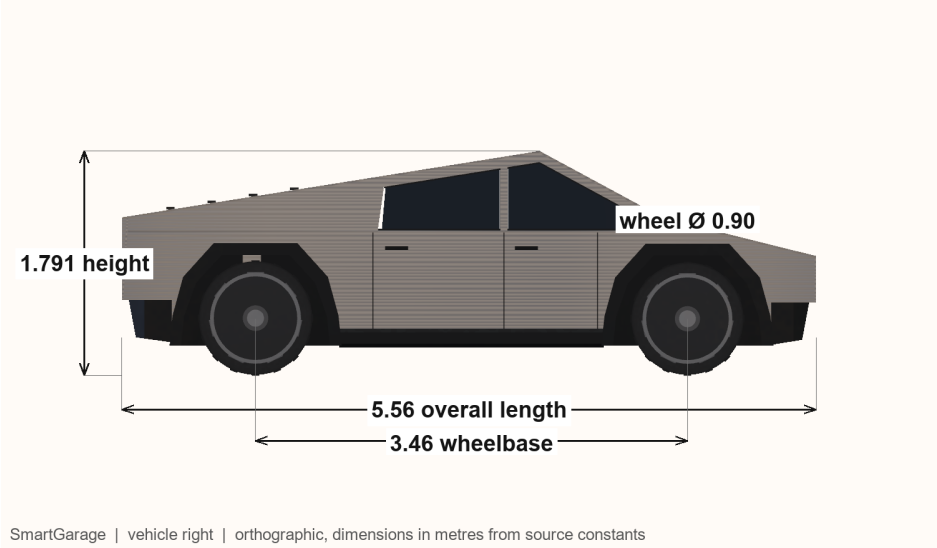
Figure 1: The four reusable primitives in wireframe (keys `5`–`8`, then `L`).

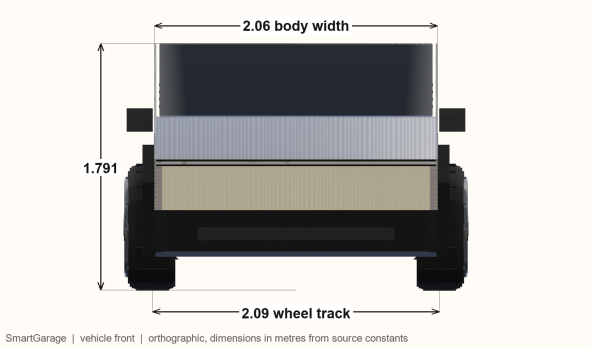
3.2 Vehicle: three-view drawing, construction sequence and labelled parts

The vehicle’s local origin is on the ground midway between the axles, nose toward $-z$. The upper silhouette is one piecewise-linear function `ctRoofLineY(z)` through measured stations: nose (z= − 2.760, y=0.951), cowl $(-1.540, 1.254)$, apex $(-0.545, 1.791)$, roof end $(0, 1.704)$, tail rail $(2.800, 1.260)$. Panels read their heights from it, so all folds meet. Figure 2 gives the orthographic views (rendered with the program’s orthographic projection; dimensions computed from the source constants).

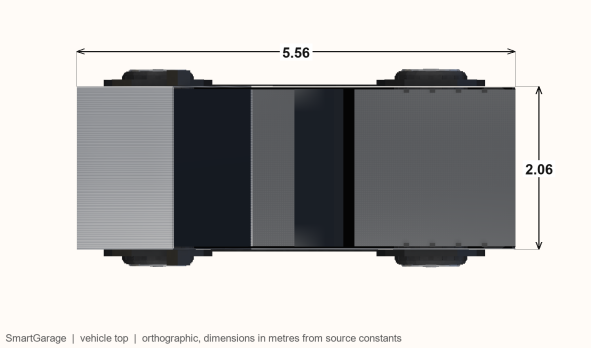
Custom vertices and normals. The hood (B) is the quad $P_1 = (-1.03, 0.951, -2.760)$, $P_2 = (1.03, 0.951, -2.760)$, $P_3 = (1.03, 1.254, -1.540)$, $P_4 = (-1.03, 1.254, -1.540)$: the nose edge is 0.303 m lower than the cowl edge over 1.22 m, which creates the $\arctan(0.303/1.22) = 13.95°$ slope. Its normal is

$$\mathbf{n} = \frac{(P_2 - P_1) \times (P_3 - P_1)}{\|(P_2 - P_1) \times (P_3 - P_1)\|} = \frac{(2.06, 0, 0) \times (2.06, 0.303, 1.22)}{\|\cdot\|} = \frac{(0, -2.513, 0.624)}{2.590} = (0, -0.971, 0.241), \tag{3}$$



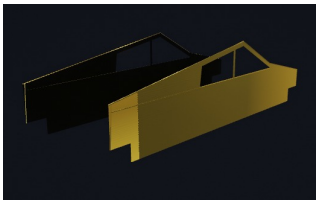


(a) Front View

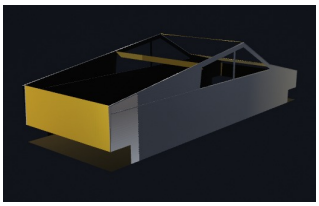


(b) Top View

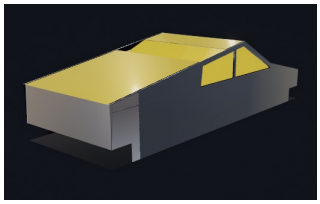
Figure 2: Vehicle three-view (Right-Side View on top, then Front and Top): length 5.56, wheelbase 3.46, body width 2.06, height 1.791, wheel Ø 0.90 (m).



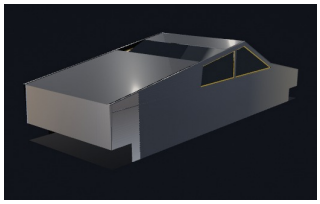
(a) 1: side skin (2 flanks)



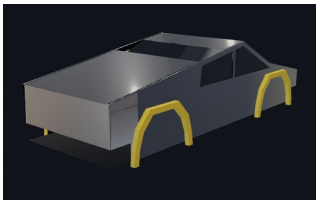
(b) 2: hood, fascia, tailgate, floor



(c) 3: windscreen, glass, roof



(d) 4: black trim, B-pillar



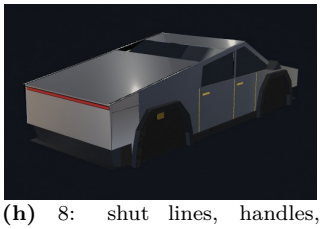
(e) 5: wheel-arch cladding



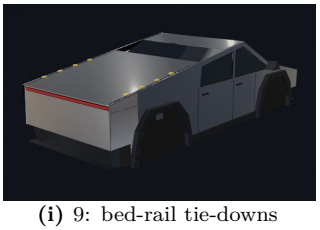
(f) 6: front and rear bumpers



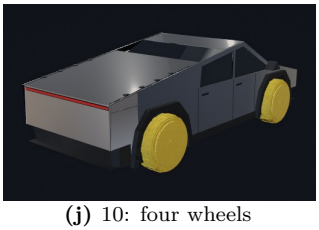
(g) 7: light bars, fascia, mirrors



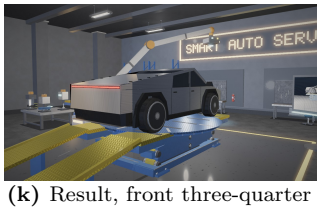
(h) 8: shut lines, handles, charge port



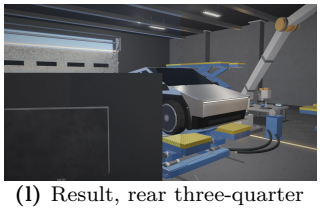
(i) 9: bed-rail tie-downs



(j) 10: four wheels

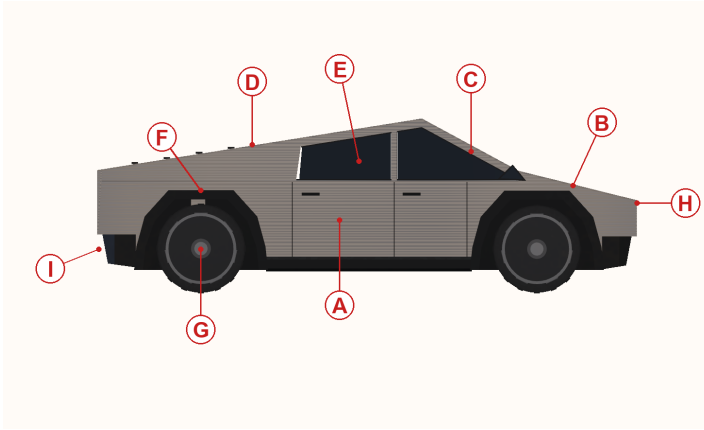


(k) Result, front three-quarter



(l) Result, rear three-quarter

Figure 3: Actual construction order of `drawCar()` (teaching view: key `2`, then `N`/`B`); amber = part added in that step.



Lbl	Part	Geometry	Function / constants
A	side skin	fan panels at $x = \pm 1.03$	`truckPushSidePanel`
B	hood	quad, 13.95°	`buildCybertruck-ShellMesh`
C	windscreen	quad cowl→apex	`CT_COWL_*`, `CT_APEX_*`
D	roof / vault cover	dark quads	`ctRoofLineY`
E	side glass	quads, x=1.006	`CT_X_GLASS`
F	arch cladding	8-point extrusion	`truckPushFive-FacetFender`
G	wheel	$TR_z(90)R_y(\phi)$	`drawWheel`
H	light bar + 2 lights	emissive quad	`headlightLocal-Position`
I	lower band	quads / boxes	bumper batches

Figure 4: Labelled right-side view and part table; every part is a child of the vehicle matrix M_{car}.

which points down, so `truckFaceNormal()` flips it to $(0, 0.971, -0.241)$ (outward). All vertices of a facet share this normal, which gives the flat-panel look. Batches are drawn with `glDrawArrays` (13 VAOs).

Wheel chain (front-left).

$$M_{FL} = M_{\text{car}}\; T(-1.045,\; 0.450,\; -1.73)\; R_z(90^\circ)\; R_y(\phi)\; S(0.90,\; 0.38w_k,\; 0.90), \tag{4}$$

where 1.045 =`CAR_FTRACK`+`CAR_WHEEL_OUTSET`, 0.450 =`CAR_FWHEEL_R`, −1.73 =`CAR_FAXLE_Z`, $R_z(90^\circ)$ lays the cylinder axis along the axle and $R_y(\phi)$ spins it (w_k = layer width factor 0.56–0.86).

IMPLEMENTATION CARD — Front-left wheel **File:** `SceneObjects.cpp` (constants), `Main.cpp` (`updateCarDrive`)
Function: `drawCar()` → `drawWheel()`
Variables = current values: `CAR_FWHEEL_R`=0.450, `CAR_FAXLE_Z`=−1.73, `CAR_FTRACK`=1.010, `gSwitches.wheelSpin`=ϕ (deg)
If changed: larger `CAR_FWHEEL_R` ⇒ bigger wheel and slower spin for the same distance; changing `CAR_FAXLE_Z` moves the axle along the body; `wheelSpin` rotates the wheel about its axle.
Equation / evidence: Eqs. (4), (26); Figs. 2–4

3.3 Hydraulic scissor lift: how it works, step by step

What it is. A scissor lift is two steel arms of equal length L crossed like an "X" and joined by a pin at their middle. On the base, one lower end is a *fixed pivot*; the other is a *roller* that slides in a track. The hydraulic ram pushes the roller toward the fixed pivot, so the arms stand up (the angle θ grows) and the upper ends, which carry the platform, rise. SmartGarage stacks **two** X's (stages) under each of the two runways, so the platform rises by $2L\sin\theta$. Left and right stacks are identical: `drawScissorStack()` is called twice, at $x = \pm 0.98$. **Step-by-step.**

1. *One arm.* An arm of length $L = 3.30$ at angle θ gains height $L\sin\theta$ and covers a horizontal span $L\cos\theta$ between its pivots.
2. *Two stages.* The upper X stands on the lower X, so the platform underside is at $y = 0.22 + 2L\sin\theta$ (0.22 is the top of the base frame).
3. *From height to angle.* The program is given a lift height h (`gLiftHeight`; keys `P`, `[ ]`, or the service). At rest the platform is already 0.58 above the frame (`LIFT_REST_H`), so $2L\sin\theta = 0.58 + h$. Solving for the angle:

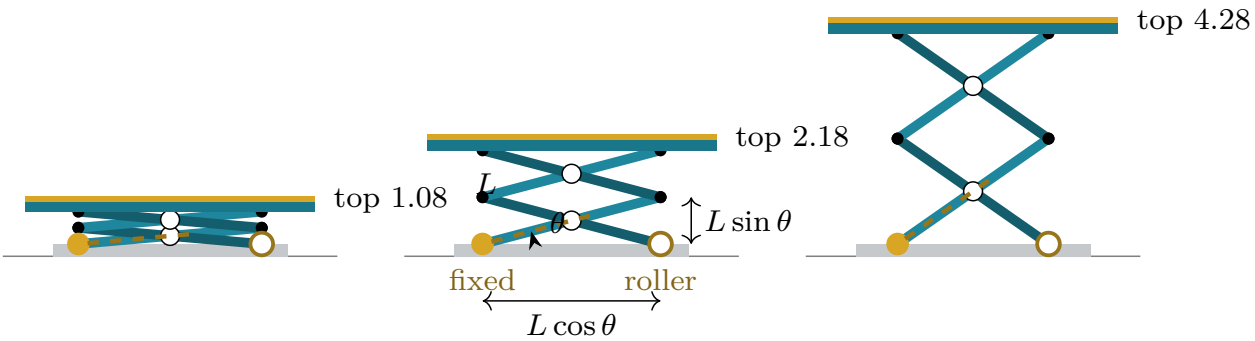
$$\theta = \arcsin\!\left(\frac{0.58 + h}{2L}\right),\; L = 3.30, \qquad \texttt{liftTopY}(h) = 0.22 + 2L\sin\theta + 0.18 + 0.10 = 1.08 + h. \tag{5}$$

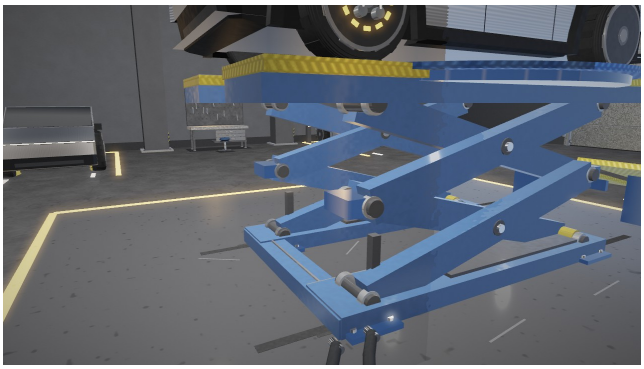
4. *The surface the tyres stand on.* Beam 0.18 and runway pad 0.10 are added, giving $y_{\text{top}} = 1.08 + h$. The vehicle matrix uses exactly this number, so raising the lift raises the car with no other change.
5. *The roller slides.* The span $L\cos\theta$ shrinks as the arms stand up, so the roller moves toward the fixed pivot: from 3.287 at $h = 0$ to 2.705 at $h = 3.2$, i.e. 0.58 m of travel (it is drawn at $(x,\, y_{\text{bottom}} + 0.02,\, +\text{halfSpan})$, $\text{halfSpan} = L\cos\theta/2$).

Table 2: Worked values of Eq. (5) (all computed from $L = 3.30$, base 0.22). $h = 1.10$ is the service height, $h = 3.20$ the `P` key target.

h (m)	$\sin\theta$	θ	$L\sin\theta$ (stage rise)	$L\cos\theta$ (span)	$2L\sin\theta$	$y_{\text{top}} = 1.08 + h$
0.00	0.0879	5.04°	0.290	3.287	0.58	1.08
0.50	0.1636	9.42°	0.540	3.256	1.08	1.58
1.10	0.2545	14.75°	0.840	3.191	1.68	2.18
2.00	0.3909	23.01°	1.290	3.037	2.58	3.08
3.20	0.5727	34.94°	1.890	2.705	3.78	4.28

How the code draws one stack. For each stage $s = 0, 1$ the bottom pivots are at $y_b = 0.22 + s\,L\sin\theta$ and the arms cross at $y_{\text{mid}} = y_b + L\sin\theta/2$. Arm A is a cube $S(0.20, 0.32, 3.30)$ rotated $R_x(-\theta)$ at $(x - 0.12,\, y_{\text{mid}},\, 0)$; arm B is the *same cube* rotated $R_x(+\theta)$ at $(x + 0.12,\, y_{\text{mid}},\, 0)$ — the mirror rotation is what makes the X, and





(a) The mechanism at $h = 1.10$ (rendered)

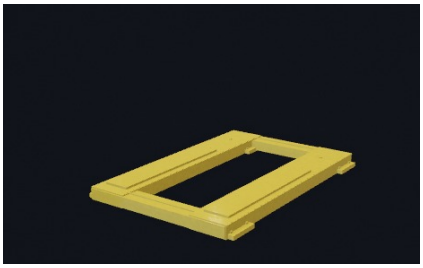


(b) The lift raised to $h = 2.0$ (rendered)

Figure 5: Side view drawn to scale at three heights (z horizontal, y vertical). Cyan = the two arms of each X, white dot = centre pin, amber disc = fixed pivot, open circle = sliding roller, dashed line = hydraulic ram, blue/amber slab = platform beam and runway pad. Below: the same lift as rendered by the program.

the 0.24 offset in x stops the two arms sharing a plane. Four pivot pins, bearing collars, a centre pin, the yellow roller and the connecting plates are placed at $z = \pm$halfSpan and $y = y_b$, $y_b + L\sin\theta$. **The ram** is a chrome rod ($0.84L$ long) inside a dark barrel ($0.52L$) lying along arm A: a part a fraction f of the way up the arm is at $P(f) = P_0 + f\,L\,(0, \sin\theta, \cos\theta)$, and because a cylinder's own axis is $+y$ it is turned by $R_x(90° - \theta)$. The ram is positioned relative to arm A, so it tilts with the arm; it is not a separate extension simulation.

Motion and safety. `gLiftHeight` moves at `kLiftSpeed`= 0.55 m/s, so the automatic LIFT_UP ($0 \to 1.10$) takes $1.10/0.55 = 2.0$ s (state time in Fig. 30); a manual move is refused by `liftPositionSafe()` if the robot arm would hit the raised platform or vehicle, or if h leaves $[0, 3.80]$.



(a) 1: base rails, tracks, anchors



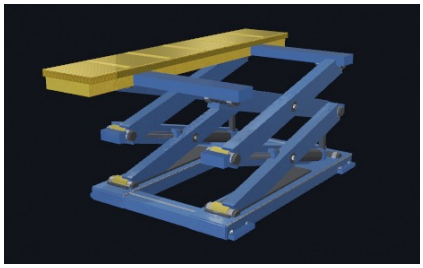
(b) 2: left scissor stack + ram



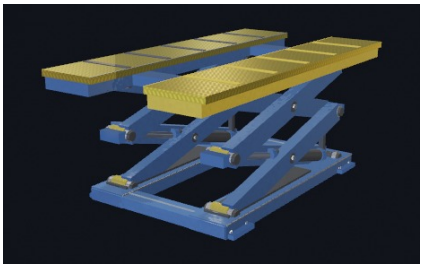
(c) 3: right scissor stack + ram



(d) 4: platform cross beams

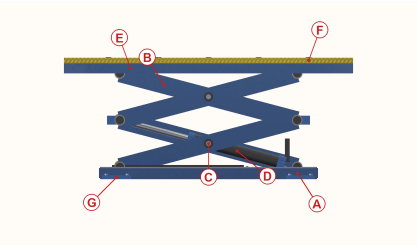


(e) 5: left runway

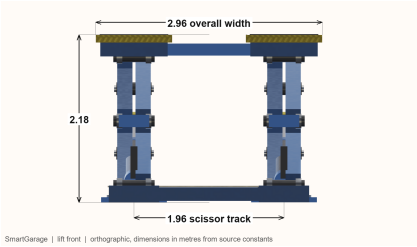


(f) 6: right runway (complete)

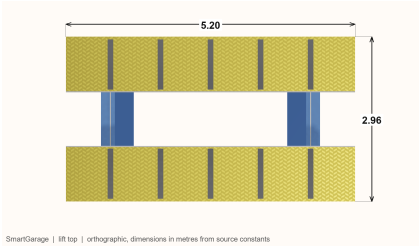
Figure 6: Actual construction order of `drawHydraulicLift()` at $h = 1.10$ (key `3`, then `N`/`B`); amber = part added in that step. Step values: base rail $S(0.66, 0.22, 3.90)$, cross members $S(2.60, 0.18, 0.34)$ at $z = \pm 1.80$, platform beams $S(2.70, 0.16, 0.34)$ at $z = \pm 1.55$, runway beam $S(0.88, 0.18, 5.20)$, pad $S(1.00, 0.10, 5.20)$.



(a) Right-Side View, parts A–G



(b) Front View



(c) Top View

Figure 7: Hydraulic lift three-view at $h = 1.10\,\mathrm{m}$. A base rail, B scissor arms, C centre pin, D ram, E platform beam, F runway pad, G anchor plate.

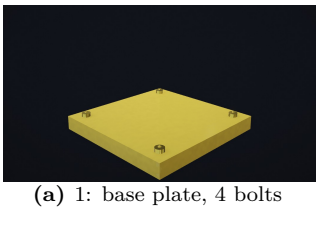
IMPLEMENTATION CARD — Scissor lift **File:** `SceneObjects.cpp`, `Main.cpp`
Function: `scissorAngle()`, `drawScissorStack()`, `drawHydraulicLift()`, `liftTopY()`
Variables = current values: `LIFT_ARM_LEN`=3.30, `LIFT_REST_H`=0.58, `LIFT_BASE_TOP`=0.22, `LIFT_TRACK`=0.98, `gLiftHeight` (0–3.80), `kLiftSpeed`=0.55 m/s, `kServiceLiftTarget`=1.10
If changed: raising `gLiftHeight` increases θ (Table 2), opens the X and carries the vehicle; larger `kLiftSpeed` raises faster; longer `LIFT_ARM_LEN` gives a smaller θ for the same height; larger `LIFT_REST_H` raises the lowest position.
Equation / evidence: Eq. (5); Figs. 5, 6, 7

3.4 Robotic arm: construction sequence and calculation

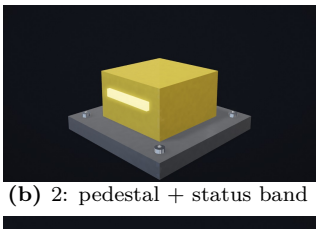
The arm is built bottom-up; every step adds parts in the frame of its parent, so one product of matrices places each part:

$$M_{\text{turn}} = T(-4.6, 0, -2.2)R_y(\gamma), \quad M_1 = M_{\text{turn}}T(0, 1.05, 0)R_z(\theta_1), \quad M_2 = M_1T(0, 4.5, 0)R_z(\theta_2),$$
$$M_{\text{wrist}} = M_2T(0, 4.0, 0)R_z(\omega)R_y(\rho), \qquad M_{\text{tool}} = M_{\text{wrist}}T(0, -0.16, 0). \tag{6}$$

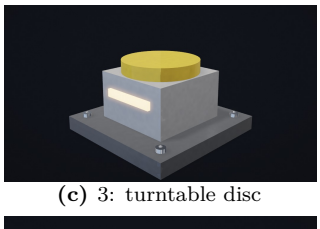
Each link is `place((0,L/2,0),(w,L,d))` in its joint frame, so its bottom sits on the joint and the joint is the pivot. **Joint ranges and rates** (`Main.cpp`): base yaw `gRobotBaseYaw` $\pm 180°$ (`T/G`, 45°/s); shoulder `gJoint1Angle` $-110° \dots 55°$ (`U/J`, 35°/s); elbow `gJoint2Angle` $-150° \dots 120°$ (`I/K`, 35°/s); wrist pitch `gWristPitch` $\pm 270°$ (`Y/H`, 55°/s); wrist roll `gWristRoll` $\pm 180°$ (← / →, 55°/s); tool `gRobotTool` 0..3 (`F1`–`F4`); HOME $= (60, -44, -30, 74, 0)$. **Worked calculation.** Forward kinematics in the turntable plane (identical to `robotSafetyPoints()`):



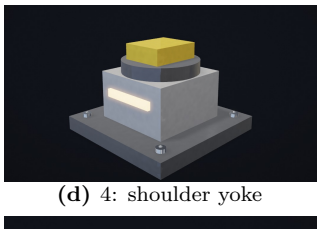
(a) 1: base plate, 4 bolts



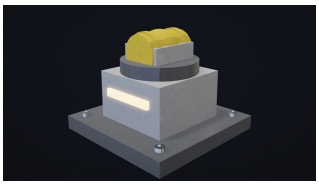
(b) 2: pedestal + status band



(c) 3: turntable disc



(d) 4: shoulder yoke



(e) 5: shoulder joint



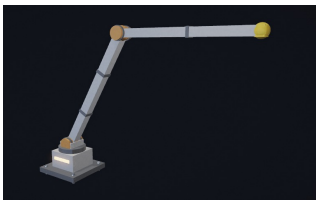
(f) 6: lower arm L_1



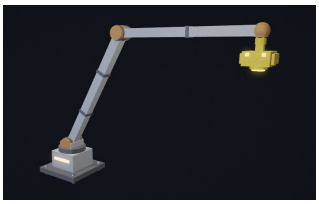
(g) 7: elbow joint



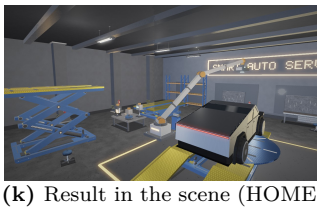
(h) 8: forearm L_2



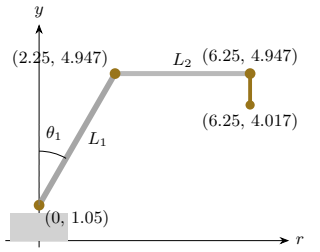
(i) 9: wrist + coupler



(j) 10: scanner tool head



(k) Result in the scene (HOME pose)

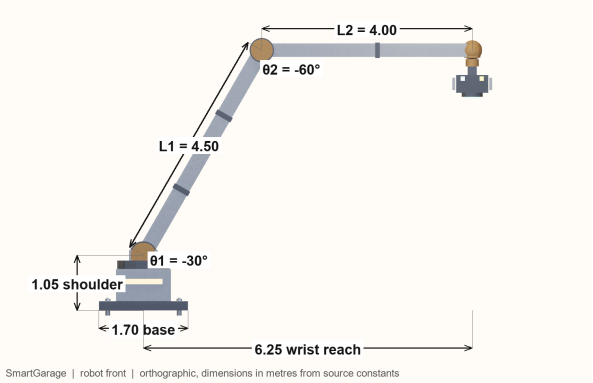


(l) Kinematic chain (see caption)

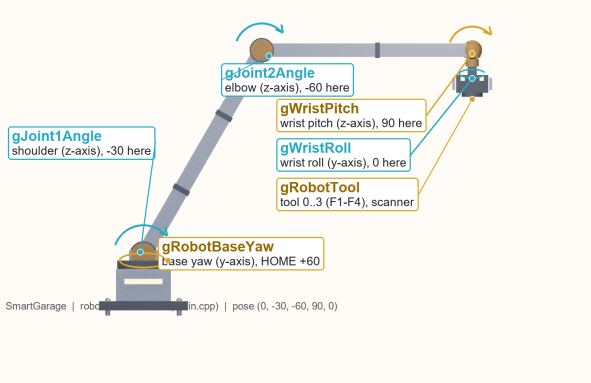
Figure 8: Actual construction order of `drawRobotArm()` (key `4`, then `N/B`) at $(\gamma, \theta_1, \theta_2, \omega) = (0, -30°, -60°, 90°)$; steps 1–5 are zoomed on the pedestal. (l) Kinematic chain of this pose: $L_1 = 4.5$, $L_2 = 4.0$; $\theta_1 + \theta_2 = -90°$ makes the forearm horizontal and $\theta_1 + \theta_2 + \omega = 0$ makes the tool hang vertically.

$$\mathbf{e} = (-L_1 \sin\theta_1,\ 1.05 + L_1 \cos\theta_1), \qquad \mathbf{w} = \mathbf{e} + \big(-L_2 \sin(\theta_1{+}\theta_2),\ L_2 \cos(\theta_1{+}\theta_2)\big),$$
$$\mathbf{t} = \mathbf{w} + \ell\big(\sin\Sigma,\ -\cos\Sigma\big), \qquad \Sigma = \theta_1{+}\theta_2{+}\omega. \tag{7}$$

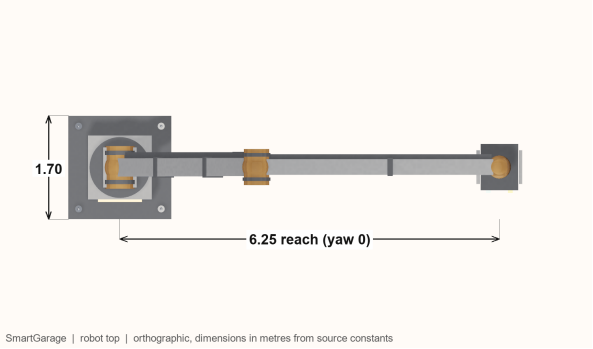
For the pose of Fig. 8: $\mathbf{e} = (4.5\sin 30°,\ 1.05 + 4.5\cos 30°) = (2.250,\ 4.947)$; $\theta_1{+}\theta_2 = -90° \Rightarrow \mathbf{w} = (2.25 + 4.0,\ 4.947 + 0) = (6.250,\ 4.947)$; $\Sigma = 0 \Rightarrow$ scanner lens ($\ell = 0.93$) at $(6.250,\ 4.017)$. Link centres (steps 6 and 8): lower arm $(1.125,\ 2.999)$, forearm $(4.250,\ 4.947)$; the two cable bands of step 6 are at $y = 1.05 + 1.44\cos 30° = 2.297$ and $1.05{+}3.06\cos 30° = 3.700$. Poses were solved offline with two-link inverse kinematics, $D = \frac{x^2+y^2-L_1^2-L_2^2}{2L_1L_2}$, $\theta_2 = \mathrm{atan2}(\pm\sqrt{1-D^2}, D)$; recomputing `POSE_LUG_1` from its wrist point gives $\theta_1 = 5.05°$, $\theta_2 = -139.01°$ exactly (reachable annulus $0.5 \le r \le 8.5$).



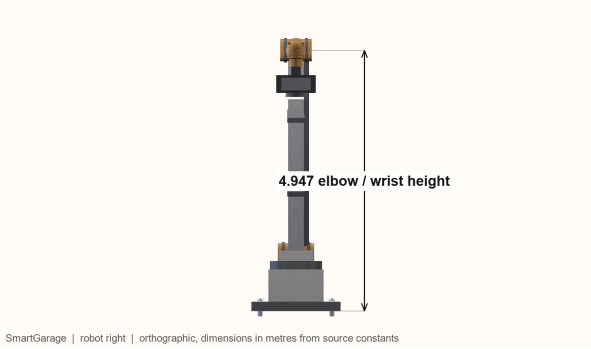
(a) Front View



(b) Joint variables and rotation axes



(c) Top View



(d) Right-Side View

Figure 9: Robot three-view at $(\gamma, \theta_1, \theta_2, \omega) = (0, -30°, -60°, 90°)$; (b) labels every joint with its `Main.cpp` variable and rotation axis (arrows show the positive rotation).

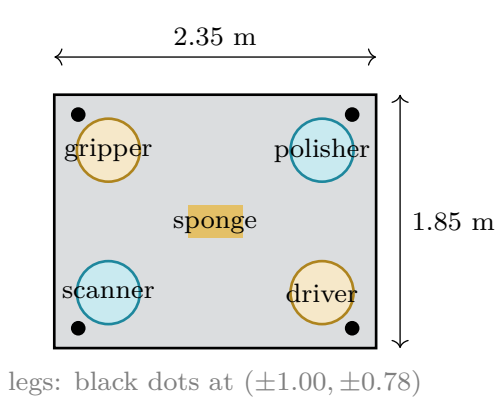
IMPLEMENTATION CARD — Robot arm (5-DOF) **File:** `SceneObjects.cpp` (geometry), `Main.cpp` (angles, poses, limits)
Function: `drawRobotArm()`, `moveRobotTowardPose()`, `processMovement()`
Variables = current values: `ARM_X,ARM_Z`=$(-4.6, -2.2)$, `ARM_SHOULDER_Y`=1.05, `ARM_LOWER_LEN`=4.50, `ARM_UPPER_LEN`=4.00, joint ranges above, `POSE_HOME`
If changed: increasing `gJoint1Angle` swings the whole chain toward $-x$ (R_z turns $+y$ toward $-x$); every child (elbow, wrist, tool, spotlight) follows because each frame is multiplied onto its parent; changing a link length moves all frames above it.
Equation / evidence: Eqs. (6), (7); Figs. 8, 9

3.5 Tool station (the table) and the tool exchange: how it works

What it is. The tool station is an open steel table beside the robot, built in the frame $T(-7.9, 0, -1.85)$ (that is the robot base $(-4.6, -2.2)$ moved by $(-3.30, +0.35)$). It has **four docks**, one per quick-change tool, in a 2×2 grid so the wrist can drop straight onto one dock without touching its neighbour. A dock is a short chrome connector post on a dark pedestal; a tool stands on it exactly as it will later sit on the robot's coupler. The gripper also picks up a loose *sponge* from the middle of the table (Fig. 10). **Numbers (`drawRobotToolStation`).** Tabletop $S(2.35, 0.11, 1.85)$ at $y = 0.78$ (top face 0.835); under-frame $S(2.20, 0.06, 1.70)$ at $y = 0.70$; four legs $S(0.11, 0.74, 0.11)$ at $(\pm 1.00, 0.37, \pm 0.78)$; two low stretchers at $y = 0.17$. Dock pedestal $S(0.46, 0.05, 0.46)$ at $y = 0.865$, ring $y = 0.905$, connector post at $y = 0.945$ whose top is the dock height 0.975 (`kToolDockTopY`). **How it behaves.** The station is told which tool is on the arm (`selectedTool`). That dock is empty, so its pedestal ring and the small status lamp in the tabletop *glow cyan*; the other three docks show their tool standing on the connector, lifted by its own drop so that its lowest point just touches it:

$$y_{\text{tool}} = 0.975 + \texttt{toolBottomDrop} \cdot k_{\text{rack}}$$
$$0.975 + 0.770 = 1.745\ \text{(scanner)}, \quad 1.965\ \text{(driver)}, \quad 1.845\ \text{(gripper)}, \quad 1.747\ \text{(polisher)}. \tag{8}$$

The world position of a dock is $T(-7.9, 0, -1.85)$ applied to its local position: $(-8.68,\ 0.975,\ -2.37)$ scanner, $(-7.12, -2.37)$ driver, $(-8.68, -1.33)$ gripper, $(-7.12, -1.33)$ polisher. **The exchange, step by step** (`runToolChange()`; every step is slow and visible): (0) swing clear of the vehicle and over the rack; (1) line up 0.70 above the dock the current tool came from; (2) descend slowly onto it; (3) pause, then *release* — the tool stays



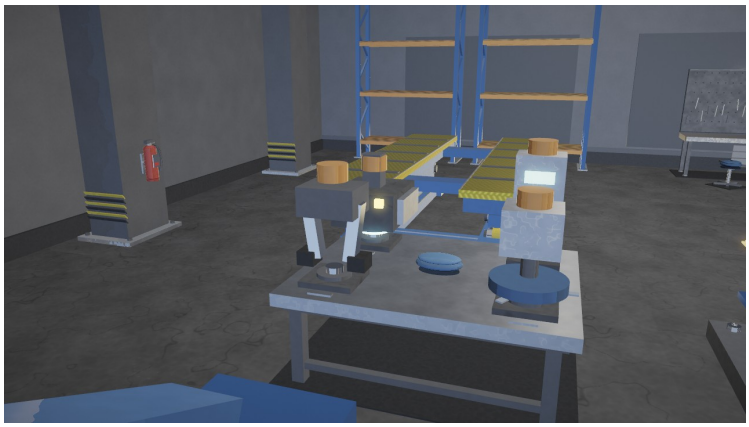
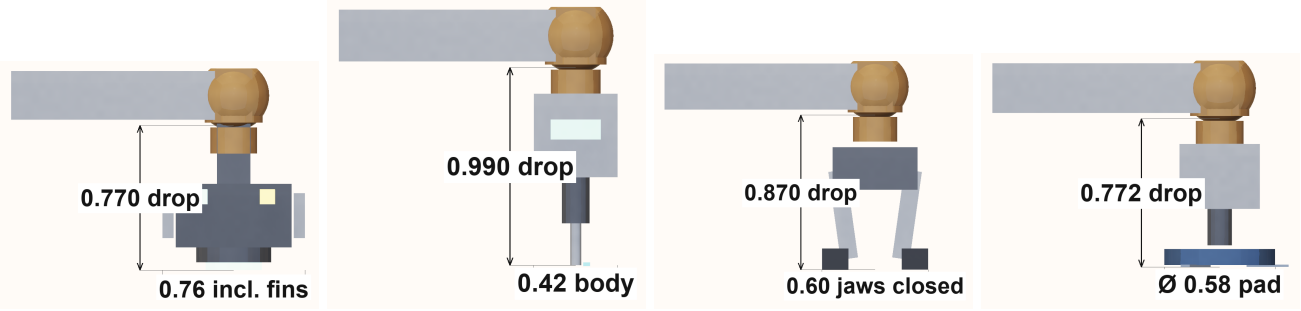


Figure 10: Tool station in plan (left, to scale, table-local x right and z down) and as rendered with all four tools docked (right). Dock positions (±0.78, 0.975, ±0.52).

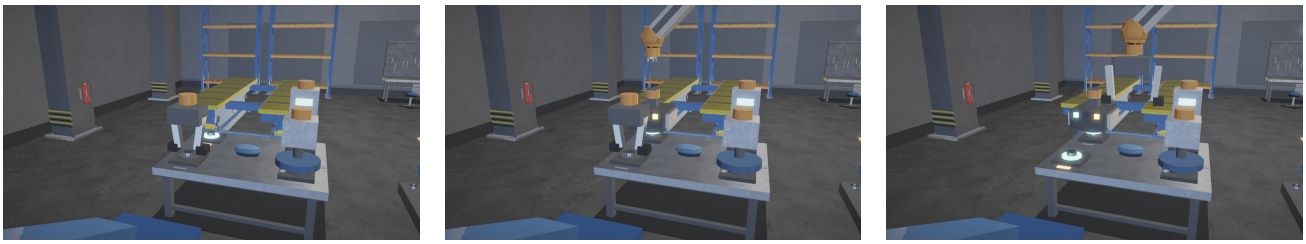
on the dock and the arm becomes a bare coupler; (4) lift clear; (5) hold 0.55 s so a bare coupler and both tools are visible together; (6) travel above the next dock; (7) descend; (8) pause, then *latch* — the new tool becomes part of the arm; (9) lift it off the dock (Fig. 12). **Dock check by forward kinematics.** For `POSE_DOCK_SEAT[0]` $= (177.61°, -23.40°, -121.56°, 144.96°)$: $\Sigma = -23.40 - 121.56 + 144.96 = 0°$ (tool vertical); $\mathbf{e} = (1.787, 5.180)$, $\mathbf{w} = (4.084, 1.905)$, $\mathbf{t} = (4.084,\ 0.975)$; yaw 177.61° gives world $x = -4.6 + 4.084\cos 177.61° = -8.680$ and $z = -2.2 - 4.084\sin 177.61° = -2.370$ — exactly the dock position above; the other three seat poses reproduce their docks to within 3 mm. Tool numbers: scanner housing 0.62×0.34×0.76 with a cyan lens at $y = -0.74$;



(a) Scanner **(b)** Impact driver **(c)** Gripper **(d)** Polisher

Figure 11: The four tool heads (front orthographic views, "drop" = `toolBottomDrop()`).

the driver socket spins R_y(driverAngle) at 980 °/s and its tip is 1.15 from the wrist; gripper fingers sit at $x = \pm(0.18 + 0.16\,o)$ with opening $o \in [0, 1]$ and tilt $\mp 8°(1 - 0.35\,o)$; the polisher pad Ø 0.58 spins at 620 °/s. A held sponge is drawn in $M_{\text{tool}}T(0, -0.91, 0)$: parenting is the attachment.



(a) $t = 74.6$ s: scanner seated **(b)** $t = 77.5$ s: bare coupler **(c)** $t = 80.5$ s: gripper takes sponge

Figure 12: Visible tool exchange during the GET_SPONGE state.

IMPLEMENTATION CARD — Tool station (table) **File:** `SceneObjects.cpp`, `Main.cpp`
Function: `drawRobotToolStation()`, `toolDockPosition()`, `runToolChange()`
Variables = current values: table 2.35×1.85, top 0.835; `kToolDockTopY`=0.975; docks (±0.78, ±0.52); frame $T(-7.9, 0, -1.85)$; `POSE_DOCK_HOVER/SEAT`; speed scales 0.24–1.25
If changed: moving `kToolDockTopY` lifts all docks (the seat poses must then be re-solved); a larger speed scale makes the exchange faster; a different drop value changes how a tool sits on the connector; changing the dock grid moves where the arm descends.
Equation / evidence: Eq. (8); Figs. 10, 11, 12

3.6 Roller garage door: how it works

What it is. A roller shutter: five flat panels (each 13.0 m wide and 0.98 m high, together the 4.9 m opening) held in two C-section guide rails. A motor turns a shaft at the top; the panels slide up the rails and then *wind round a drum* above the opening, just like a real shutter. The only animated number is the **door offset** d (`gDoorOffset`, metres): $d = 0$ closed, $d = 4.9$ open. Keys `O`, `Z/X/C` and the state machine only change the target `gDoorTarget`; `moveToward()` then moves d at `kDoorSpeed`= 0.8 m/s, so a full opening takes 4.9/0.8 = 6.125 s

(the DOOR_OPEN_IN and DOOR_OPEN states in Fig. 30). While it moves, an amber warning beacon pulses.
Step-by-step (what happens to panel $i = 0..4$, bottom to top).

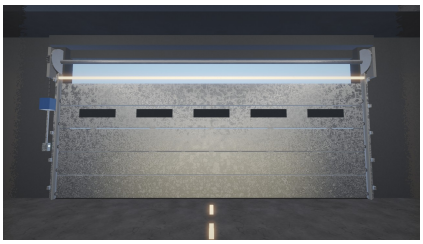
1. *Straight travel.* If it only slid up, the centre of panel i would be at $\text{travel}_i = 0.98\,(0.5 + i) + d$.
2. *How far past the top?* $\text{over} = \text{travel}_i - 4.9$. While $\text{over} \le 0$ the panel is flat in the rails: its matrix is just $T(0, \text{travel}_i, z_d)$.
3. *Winding.* When $\text{over} > 0$ the extra travel is turned into an angle round the drum shaft (arc length divided by radius): $\alpha = \text{over}/R$ with drum radius $R = 0.52$. One panel (0.98 long) therefore uses $0.98/0.52 = 1.88$ rad $\approx 108°$ of drum, and the five panels wrap about one and a half turns.
4. *Layers.* Each full turn lies on top of the last, so the radius grows by one panel thickness (0.095) per revolution: $r = R + 0.095\,\alpha/360°$. The angle is capped at 560° (drum full).
5. *Placing the panel.* Rotate about the shaft, step out to the radius, then lay the panel's length along the circumference (the shaft lies along x, so the rotations are about x):

$$\alpha = \frac{\text{over}}{R}, \qquad r = R + 0.095\frac{\alpha}{2\pi}, \qquad M_{\text{panel}} = T(0, 5.85, z_d)\, R_x(\alpha)\, T(0, -r, 0)\, R_x(90°), \tag{9}$$

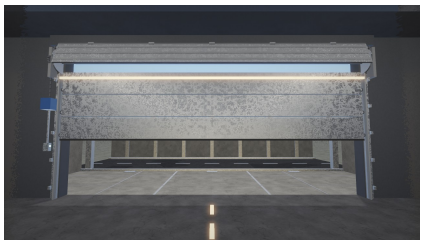
where $5.85 = \text{DOOR_H} + 0.95$ is the height of the drum centre and $z_d = 12.88$ the door plane.

Table 3: Door state for five offsets d (computed from Eq. (9)): "flat" = still in the rails, otherwise the winding angle α of that panel.

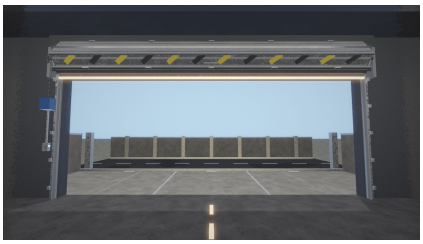
d (m)	panel 0	panel 1	panel 2	panel 3	panel 4
0.00	flat	flat	flat	flat	flat
1.50	flat	flat	flat	3.3°	111.3°
2.45	flat	flat	flat	108.0° ($r = 0.548$)	216.0° ($r = 0.577$)
3.50	flat	7.7°	115.7°	223.7°	331.7°
4.90	54.0°	162.0°	270.0°	377.9°	485.9° ($r = 0.648$)



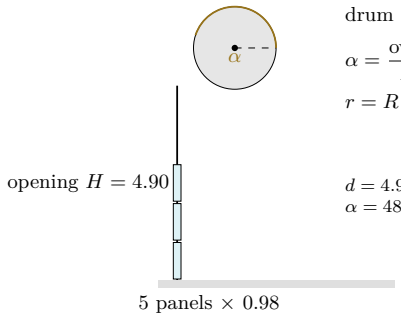
(a) $d = 0$: closed



(b) $d = 2.45$: panels 3, 4 winding



(c) $d = 4.9$: open, rolled on drum



The same door as seen from inside at three openings. Below: the winding geometry of Eq. (9); a panel that has travelled *over* the top of the opening is rotated by α about the drum shaft at the winding radius r.

Figure 13: Roller door at three openings and its winding geometry.

IMPLEMENTATION CARD — Roller door **File:** `SceneObjects.cpp`, `Main.cpp`
Function: `drawGarageDoor()`; `moveToward()` in the main loop; keys `O`, `Z/X/C`
Variables = current values: `DOOR_W`=13.0, `DOOR_H`=4.9, `DOOR_PANELS`=5, `DOOR_PANEL_H`=0.98, `DRUM_R`=0.52, drum $y = 5.85$, `kDoorSpeed`=0.80, `gDoorOffset` 0–4.9
If changed: a larger `gDoorOffset` lifts every panel and raises each winding angle; a larger `DRUM_R` winds more gently (smaller α for the same travel); more panels of lower height make a smoother roll; a larger `kDoorSpeed` opens faster.
Equation / evidence: Eq. (9); Table 3; Fig. 13

3.7 Approach ramp (the slope) and turntable

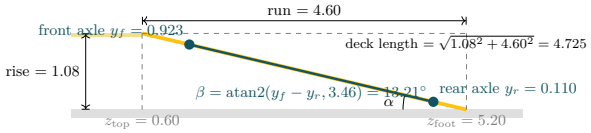
Why a ramp. The lift platform rests 1.08 m above the floor, so tyres cannot simply drive on; a drive-on ramp turns the step into a slope. The ramp is bolted to the *floor*, not to the lift, so it stays put when the platform rises, and its slope comes from the same number that places the vehicle (`liftTopY(0)`). **Step-by-step.**

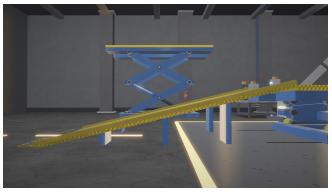
1. *Rise and run.* Rise $= \text{liftTopY}(0) = 0.22 + 0.58 + 0.18 + 0.10 = 1.08$; run $= 4.60$ (`LIFT_RAMP_LEN`).
2. *Slope angle.* $\alpha = \arctan(1.08/4.60) = 13.21°$, i.e. $\tan\alpha = 0.2348$, a 23.5% grade — a normal drive-on ramp; a shorter run would be a kerb, not a ramp.
3. *Deck length.* The sloped deck is the hypotenuse $\ell = \sqrt{1.08^2 + 4.60^2} = 4.725$, not the horizontal run.

4. *Placing the cube.* A sloped slab is a flat cube rotated about x. Rotating a box about its centre moves its ends, so the centre is put at the *mid-height* $1.08/2 = 0.54$ and mid-run $z = z_{\text{top}} + \text{run}/2 = 0.60 + 2.30 = 2.90$ ($z_{\text{top}} = \text{LIFT_Z} + \text{LIFT_PAD_L}/2 = -2 + 2.6$); $R_x(+\alpha)$ drops the $+z$ end to the floor, so both ends land exactly.
5. *Support legs.* Three legs per side stand under the deck at $t = \frac{1}{4}, \frac{1}{2}, \frac{3}{4}$ of the run with heights $1.08(1-t) = 0.81,\ 0.54,\ 0.27$.

$$\alpha = \arctan\frac{1.08}{4.60} = 13.21^\circ, \qquad \ell = \sqrt{1.08^2 + 4.60^2} = 4.725,$$
$$M_{\text{deck}} = T(\pm 0.98,\ 0.54,\ 2.90)\, R_x(13.21^\circ)\, S(1.00,\ 0.075,\ 4.725). \tag{10}$$

The car on the slope. With the vehicle centre at $z = 3.0$: front axle $z = 1.27 \Rightarrow y_f = 1.08\frac{5.2-1.27}{4.6} = 0.923$, rear axle $z = 4.73 \Rightarrow y_r = 0.110$, so the body drops $\Delta y = \frac{y_f + y_r}{2} - 1.08 = -0.564$ and pitches $\beta = \text{atan2}(0.813, 3.46) = 13.21^\circ$ — both tyres sit on the deck, and the pitch equals the ramp angle. **Turntable.** On top of the runways a disc $\varnothing 4.20$ ($R = 2.10$) is raised 6 mm above the runway top ($y = \texttt{liftTopY} + 0.006$, to avoid z-fighting). It turns the vehicle ($\psi = \texttt{carYaw}$, 34°/s) so the arm can work both flanks; the vehicle's corners sweep $\sqrt{2.84^2 + 1.03^2} = 3.02$, so the arm lifts clear before the plate turns. Index mark k is at $T(0, y, -2)R_y(\psi + 45^\circ k)\, T(1.76, 0, 0)$.





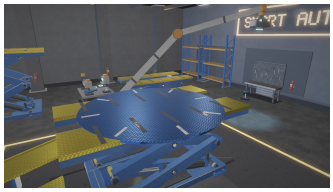


Figure 14: Ramp profile of Eq. (10) with the vehicle at $z = 3.0$ (left); rendered ramp (middle) and turntable at $\psi = 40^\circ$ (right).

IMPLEMENTATION CARD — Approach ramp (slope) and turntable **File:** `SceneObjects.cpp`, `Main.cpp`
Function: `drawLiftApproachRamp()`, `drawScene()`, `moveCarYawToward()`
Variables = current values: `LIFT_RAMP_LEN`=4.60, rise 1.08, slope 13.21°, deck 4.725, `LIFT_TRACK`=0.98, `TURN_D`=4.20, `carYaw`, 34°/s
If changed: a longer `LIFT_RAMP_LEN` lowers the slope α (gentler but longer); a taller platform makes it steeper; a larger `TURN_D` widens the plate (it must stay within the lift base); a larger turn rate spins the car faster.
Equation / evidence: Eq. (10); Fig. 14

3.8 Bézier hydraulic hoses

Each hose is a tube swept along a cubic curve (the project's only equation-defined geometry):

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t)t^2 P_2 + t^3 P_3,$$
$$B'(t) = 3(1-t)^2(P_1 - P_0) + 6(1-t)t(P_2 - P_1) + 3t^2(P_3 - P_2). \tag{11}$$

The curve is sampled at 14 points; each chord $B(t_{i-1}) \to B(t_i)$ becomes a smooth cylinder aimed by $b = \arccos d_y$, $a = \text{atan2}(d_z, -d_x)$ (d = unit chord direction), and a sphere at each knot hides the joints. For hose 0, bow $= 0.55 + 0.30\,\text{hash01}(3) = 0.727$ and the control points of Fig. 15 give $B(0.5) = 0.125P_0 + 0.375P_1 + 0.375P_2 + 0.125P_3 = (1.625,\ 0.107,\ -0.705)$, and the polyline length is 3.12 m. For the chord $t \in [0.5, 0.571]$ the direction is almost $+x$, so $b \approx 91.7^\circ$, $a = 180^\circ$: the cylinder is laid along the hose by $R_y(180^\circ)R_z(91.7^\circ)$.

IMPLEMENTATION CARD — Bézier hose **File:** `SceneObjects.cpp`
Function: `drawLiftHydraulicHose()`, `drawBezierHose()`, `drawTubeSegment()`
Variables = current values: 14 segments, radius 0.045, $P_0 = (0.20, 0.236, \mp 0.16)$, $P_3 = (3.05, 0.14, \mp 0.16)$, `bow`=0.727
If changed: raising the control points' y lifts the sag; a larger `bow` pushes the loop outward; more segments give a smoother tube at more draw calls.
Equation / evidence: Eq. (11); Fig. 15

3.9 Garage shell and furniture: workbench, work stool, racking, oil drums

The shell. Every shell part is one scaled cube and all sizes are constants (`GARAGE_W`=28, `GARAGE_D`=26, `GARAGE_H`=7, `WALL_T`=0.4):

$$M_{\text{floor}} = T(0, -0.15, 0)\, S(28.8, 0.30, 26.8), \qquad M_{\text{back wall}} = T(0, 3.5, -13.2)\, S(28.8, 7.0, 0.4),$$
$$M_{\text{post}} = T(\pm 10.45, 3.5, 13.2)\, S(7.9, 7.0, 0.5), \qquad 7.9 = \tfrac{1}{2}(28.8 - 13),\ \ 10.45 = \tfrac{13}{2} + \tfrac{7.9}{2}. \tag{12}$$

The floor top is at $y = 0$ because the centre is -0.15 for thickness 0.30; the posts leave exactly the 13 m door opening. Outside, a 20×6 m apron where the car waits is followed by an 8 m road with centre and edge lines, boundary walls with piers on both sides and at the back, and open ground to the horizon. There are no street lamps; key `0` switches the exterior between constant daylight and night. **Furniture and props** use

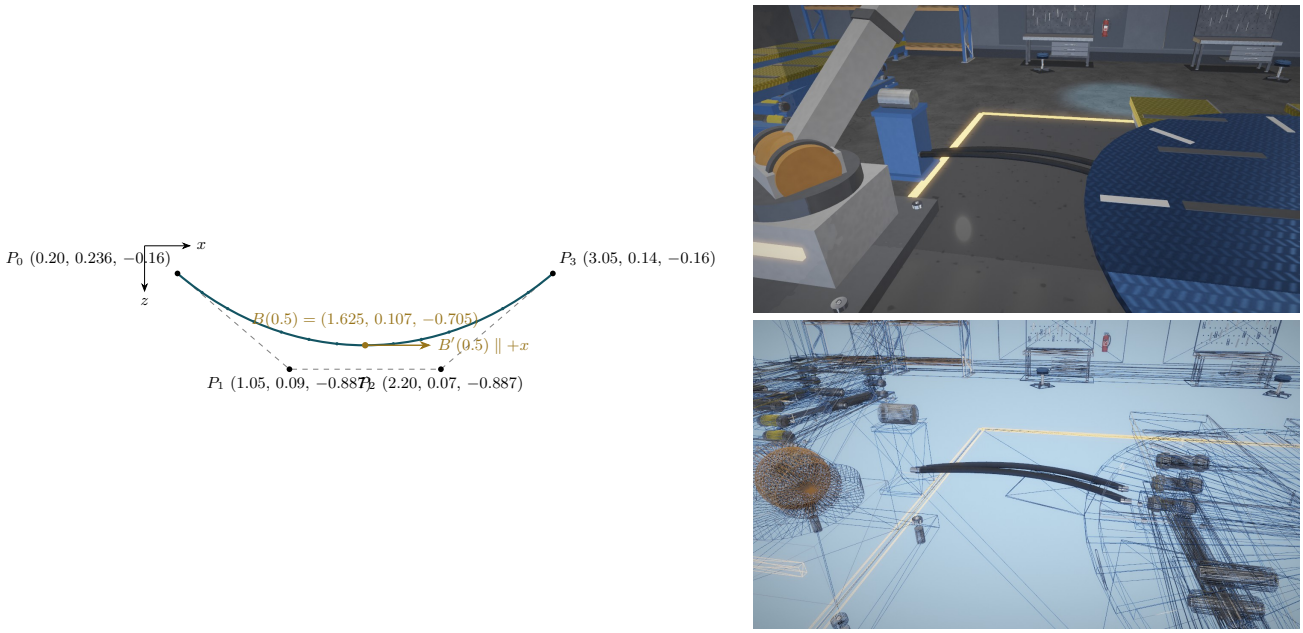
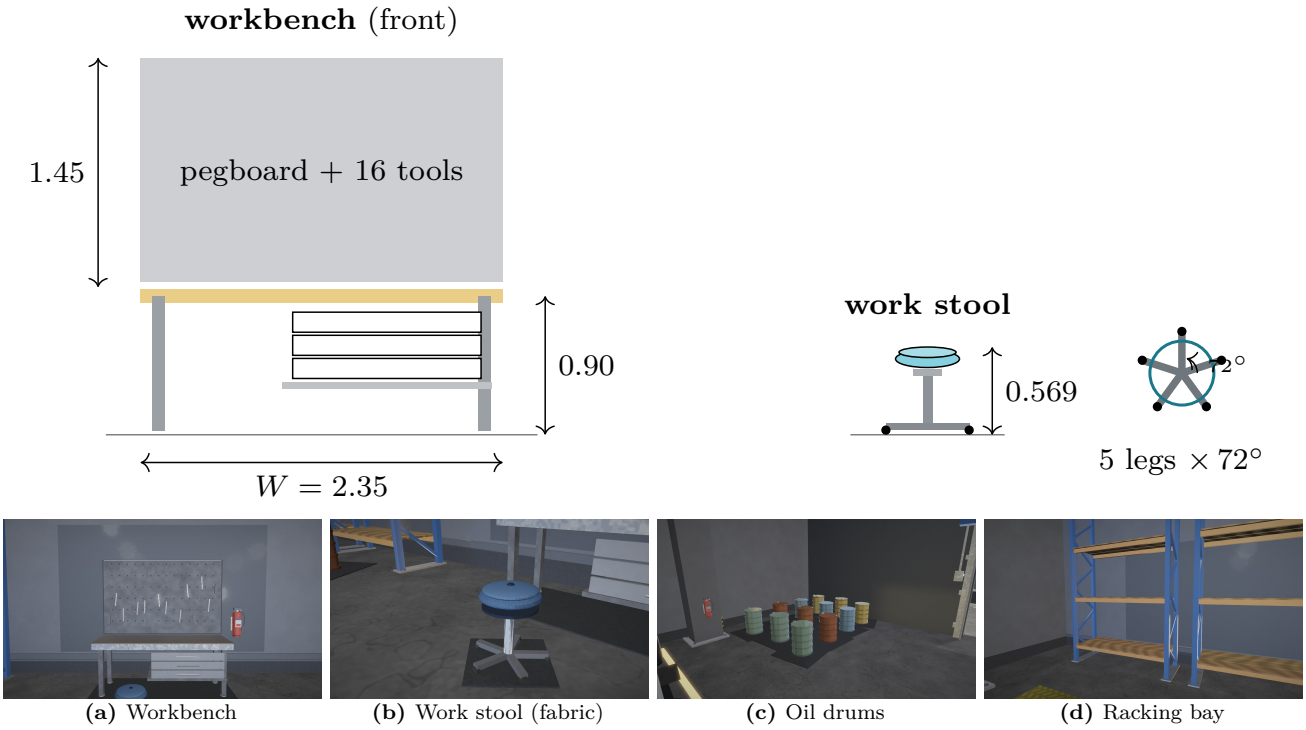


Figure 15: Control polygon and samples of hose 0 with $B(0.5)$ and its tangent (left); rendered hoses, shaded and wireframe (right).



(a) Workbench **(b)** Work stool (fabric) **(c)** Oil drums **(d)** Racking bay

Figure 16: Dimensions (metres, to scale) of the workbench front and the work stool side and top view, taken from `drawWorkbench` and `drawWorkStool`; below, the rendered props (the oil drums beside the inside entrance wall).

the same recipe: choose a primitive, scale it to the real size, translate it to its place (Fig. 16). **Workbench** (`drawWorkbench`, three in the garage). A timber top $S(2.35, 0.085, 0.78)$ at $y = 0.90$ on four legs $S(0.08, 0.87, 0.08)$ placed at $x = \pm W \cdot 0.45 = \pm 1.0575$, $z = \pm D \cdot 0.35 = \pm 0.273$; a lower shelf $S(1.36, 0.045, 0.67)$ at $y = 0.32$ and three drawers $S(1.22, 0.13, 0.03)$ at $y = 0.43$, 0.58, 0.73 under the right-hand end. Behind it a pegboard $S(2.35, 1.45, 0.04)$ is centred at $y = 0.90 + 0.82 = 1.72$ (spans 0.995–2.445) with a 14×6 grid of peg holes at $x = (c - 6.5) \cdot 0.155$, $y = 1.25 + 0.225\,r$, and 16 hanging tools at $x = (i - 7.5) \cdot 0.142$ whose heights are varied by a deterministic hash.

Work stool (`drawWorkStool`, four in the garage; the "chair"). Seat height: two flattened UV spheres, the lower cushion $S(0.415, 0.112, 0.415)$ at $y = 0.492$ (semi-axes 0.2075, 0.056) and the upper cushion $S(0.370, 0.068, 0.370)$ at $y = 0.535$, so the top of the seat is $0.535 + 0.034 = 0.569$ m. Under the seat: a seat plate $\varnothing 0.185$ at $y = 0.405$, a chrome gas post $\varnothing 0.07$ from 0.035 to 0.405, and **five legs** $S(0.27, 0.04, 0.048)$ at $(0.145, 0.055, 0)$, each rotated $R_y(72°i)$ with $72° = 360°/5$, ending in a rubber castor at radius 0.27 (footprint $\approx \varnothing 0.60$). The cushion uses the *woven-fabric* material (`MAT_FABRIC`): fully diffuse (roughness 0.97), a thread weave of $\sin(420(x + 0.7y)) \cdot \sin(420(z - 0.7y))$ (a 1.5 cm repeat measured in world metres, faded out when a pixel is wider than half a thread), a fibre fuzz and a piped seam with a tufting button — so it reads as soft upholstery, not plastic.

Oil drums (`drawOilDrum`, twelve). Each drum is one smooth cylinder with pressed ribs and rims. All twelve are stored as a staggered 3×4 block *beside the inside entrance wall* ($x = 10.0$–12.6, $z = 8.5$–12.4), outside the 13 m door opening, so the drive path, the lift and the rear work zone stay clear.
Racking bay (`drawRackingBay`). Width 2.70, depth 1.05, level spacing 1.32: height $H = 1.32n + 0.30$ (4.26 for $n = 3$), uprights at $x = \pm 1.35$, $z = \pm 0.399$, beams at $y = 0.45 + 1.32L = 0.45$, 1.77, 3.09, diagonal bracing at $\pm 38°$.

IMPLEMENTATION CARD — Workbench **File:** `SceneObjects.cpp`
Function: `drawWorkbench(parent, index)`; placed in `drawScene()` at $(-3.0, 0, -12.1)$, $(2.7, 0, -12.1)$, $(12.72, 0, -1.6)$
Variables = current values: $W = 2.35$, $D = 0.78$, `TOP_Y`=0.90, legs ± 1.0575, ± 0.273, pegboard centre $y = 1.72$ (1.45 high), 16 tools
If changed: raising `TOP_Y` lifts the top (legs and pegboard follow because they are written relative to it); a larger W widens the top, leg spacing and pegboard together; changing the hash seed re-arranges the hanging tools.
Equation / evidence: Fig. 16

IMPLEMENTATION CARD — Work stool (chair) **File:** `SceneObjects.cpp`, `Shader.cpp`
Function: `drawWorkStool(parent, index)`; fabric branch `uMaterialType == 21` in `materialMaps()`
Variables = current values: seat $S(0.415, 0.112, 0.415)$ at 0.492 and $S(0.370, 0.068, 0.370)$ at 0.535 (top 0.569), gas post $\varnothing$ 0.07, 5 legs $\times 72°$ of length 0.27, roughness 0.97, weave repeat 1.5 cm
If changed: moving the cushion y changes the seat height (the post length must follow); more or fewer than 5 legs needs $360°/n$; lowering the roughness would make the cushion shiny again; the weave frequency (420) sets the thread size.
Equation / evidence: Fig. 16

4 Transformations, Camera and Projection

Model matrix. Points are homogeneous, $\mathbf{p}' = M\mathbf{p}$. `Scene::place(pos,size,rot)` builds

$$M = T\,R_y R_x R_z\,S, \quad T = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix},\ S = \mathrm{diag}(s_x, s_y, s_z, 1),\ R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c & -s & 0 \\ 0 & s & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{13}$$

(R_y, R_z analogous; `Math3D.h` stores column-major). Read right-to-left: scale the unit cube, rotate it about its own centre, then translate (Fig. 17). *Example* (floor): $M = T(0, -0.15, 0)S(28.8, 0.3, 26.8)$ maps the corner $(0.5, 0.5, 0.5)$ to $(14.4, 0, 13.4)$: the slab top lies on $y = 0$.

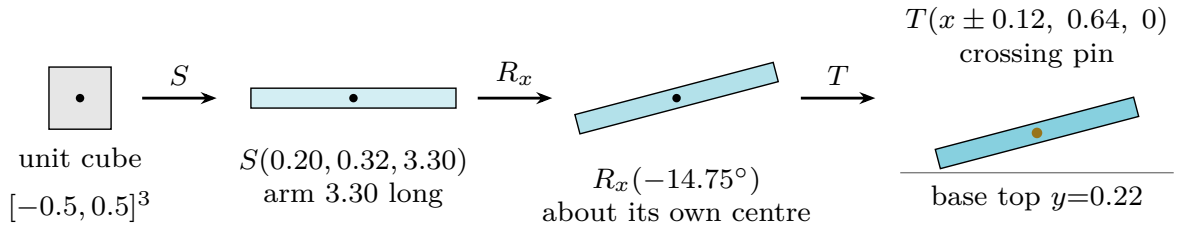


Figure 17: Unit cube → scale → rotate → translate for one scissor arm at $h = 1.10$ m.

Hierarchy. $M_{\text{child,world}} = M_{\text{parent,world}} M_{\text{child,local}}$ (Fig. 18). The vehicle matrix (`serviceCarMatrix()`) is

$$M_{\text{car}} = T(0,\ \texttt{liftTopY}(h) + \Delta y,\ -2 + 18.5\,c)\ R_x(\beta)\ R_y(\psi), \tag{14}$$

$c =$ `carDrive` (1 outside, 0 on lift), β ramp pitch, ψ turntable yaw. Changing c moves the whole car and every child with one number.

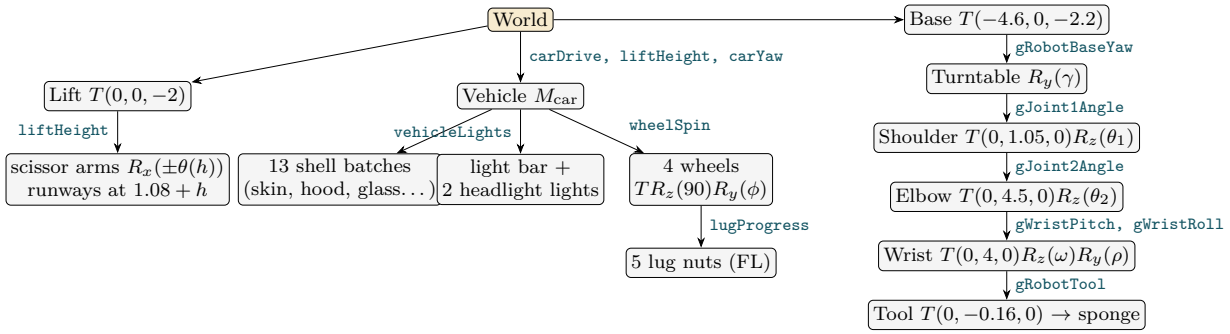


Figure 18: Parent–child transformation trees; blue labels are the variables that drive each edge.

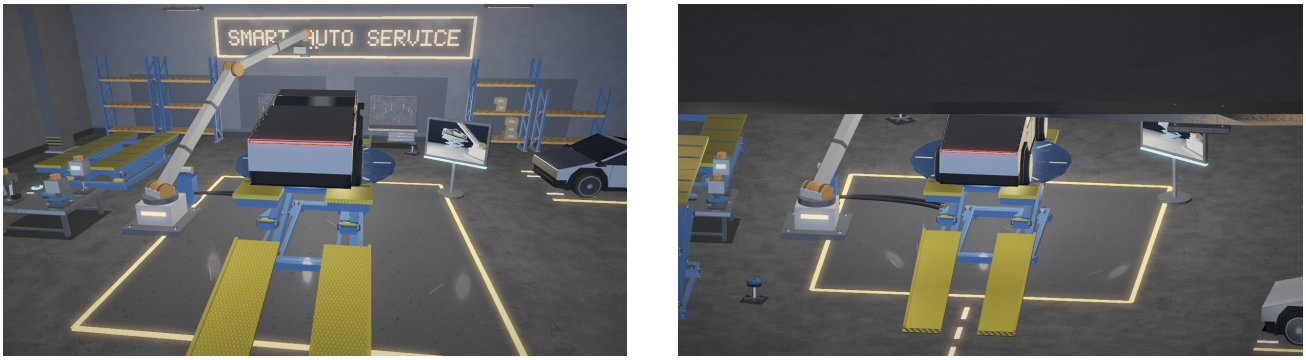
Normal matrix. Under non-uniform scale, normals use $N = (M_{3\times3}^{-1})^T$ (`normalMatrix()`); for the floor $N = \mathrm{diag}(1/28.8, 1/0.3, 1/26.8)$, and the top normal stays $(0, 1, 0)$ after normalisation.

Camera / view. $\mathbf{f} = (\cos\psi\cos\varphi, \sin\varphi, \sin\psi\cos\varphi)$, $\mathbf{r} = \widehat{\mathbf{f} \times \mathbf{y}}$; `lookAt(eye, eye+f, up)` moves the world so the eye is at the origin looking down $-z$. Speed 7 m/s (`moveSpeed`), mouse 0.1 °/px, FOV 20–90° by scroll.

Projection. Perspective ($f = 1/\tan(\text{fov}/2)$, $a = \text{aspect}$, $z_n = 0.05$, $z_f = 200$) and orthographic:

$$P = \begin{bmatrix} f/a & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{z_f+z_n}{z_n-z_f} & \frac{2z_f z_n}{z_n-z_f} \\ 0 & 0 & -1 & 0 \end{bmatrix}, \qquad O = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{z_f-z_n} & -\frac{z_f+z_n}{z_f-z_n} \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{15}$$

With fov 60°, $f = 1.732$: a point at $x = 1$ maps to $x_{ndc} = 0.195$ at depth 5 and 0.049 at depth 20 (smaller when farther). `Backspace` switches to O with $t = d\tan(\text{fov}/2)$ at the vehicle distance d, so only the projection changes (Fig. 19).



(a) Perspective: runway edges converge **(b)** Orthographic: edges stay parallel

Figure 19: Projection switch (`Backspace`): same camera position, direction and field of view; only the projection matrix changes.

IMPLEMENTATION CARD — Camera and projection **File:** `Math3D.h`, `Camera.h`, `Main.cpp`
Function: `perspective()`, `orthographic()`, `lookAt()`, `Camera::viewMatrix()`, render loop in `main()`
Variables = current values: `gCamera.fov`=60° (20–90), near 0.05, far 200, `moveSpeed`=7.0, `mouseSens`=0.10, start (4.6, 2.9, 10.2), yaw −104°, pitch −8°, `gOrthographic`
If changed: smaller fov zooms in (larger f); a larger near plane clips close objects; far plane < scene size clips the far wall; `gOrthographic`=true removes the perspective division.
Equation / evidence: Eq. (15); Fig. 19

5 Lighting, Shading, Materials and Textures

5.1 Phong lighting model (course theory, as implemented)

$$I = I_a + I_d + I_s, \qquad I_a = k_a L_a, \qquad I_d = k_d L_d \max(\mathbf{N}\cdot\mathbf{L}, 0), \qquad I_s = k_s L_s \max(\mathbf{R}\cdot\mathbf{V}, 0)^n$$

ambient: base light on every surface | diffuse: depends on the angle between normal and light | specular: highlight, sharper for larger n

`phongReflection()` in `Shader.cpp` implements the classical Phong reflection model [3] for every light j (directional, up to 10 point lights, spotlight):

$$I = \underbrace{k_a L_a}_{\text{ambient}} + \sum_j A_j \Big[\underbrace{k_d L_j \max(\mathbf{N}\cdot\mathbf{L}_j, 0)}_{\text{diffuse}} + \underbrace{k_s L_j \max(\mathbf{R}_j \cdot \mathbf{V}, 0)^n}_{\text{specular}} \Big], \qquad \mathbf{R}_j = \text{reflect}(-\mathbf{L}_j, \mathbf{N}). \tag{16}$$

Project values: $k_a L_a = 0.038$; k_d = material albedo (diffuse reduced by 0.30–0.35 m for metals); $k_s = \text{mix}(0.12, 0.75, m)$ (directional), $\text{mix}(0.08, 0.48, m)$ (point), $\text{mix}(0.10, 0.55, m)$ (spot); shininess $n = \text{mix}(180, 10, r)$ from roughness r; point attenuation $A = 1/(1 + 0.075d + 0.018d^2)$; final colour $C = k_d \odot I_{a+d} + I_s \cdot \text{mix}(1, k_d, m)$.

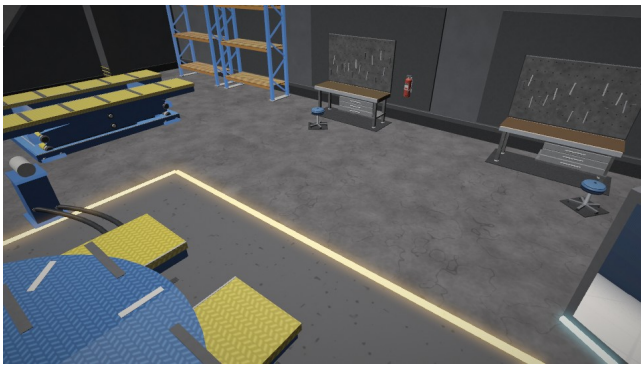
```
float n = mix(180.0, 10.0, clamp(rough, 0.04, 1.0)); // shininess
diffuse = vec3(0.038); // ambient ka*La
vec3 L = normalize(-uDirLightDirection); vec3 R = reflect(-L, N);
diffuse += Ldir * max(dot(N, L), 0.0) * (1.0 - 0.35 * metal); // kd*Ld*max(N.L,0)
specular += Ldir * pow(max(dot(R, V), 0.0), n) * mix(0.12, 0.75, metal); // ks*Ls*max(R.V,0)^n
```

Example: a plastic surface ($m = 0$, $r = 0.58$) has $n = \text{mix}(180, 10, 0.58) = 81.4$ and $k_s = 0.12$ for the sun; a point light 6.59 m away is attenuated to $A = 1/(1 + 0.494 + 0.782) = 0.44$.

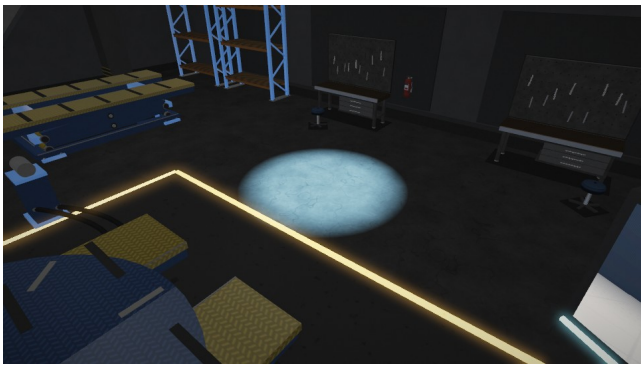
5.2 Gouraud versus Phong shading

Gouraud (`F5` once): Eq. (16) is evaluated in the *vertex* shader, which outputs `vGouraudDiffuse` and `vGouraudSpecular`; the rasteriser interpolates them: $C_f = \alpha C_1 + \beta C_2 + \gamma C_3$ (barycentric, $\alpha + \beta + \gamma = 1$) [4].

Phong (`F5` twice): the *normal* is interpolated and the same function runs per *fragment*, with shadows. The floor slab is only two triangles, so under Gouraud a spotlight or headlight pool that falls between its vertices disappears, while Phong reproduces it (Fig. 20).



(a) Gouraud: spotlight pool lost



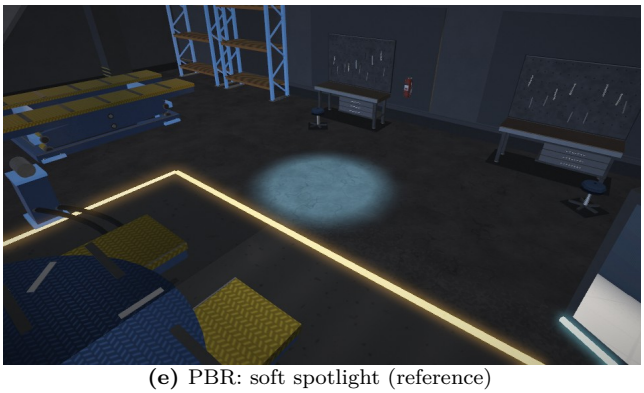
(b) Phong: crisp spotlight pool



(c) Gouraud: no headlight falloff



(d) Phong: headlight point-light pool



(e) PBR: soft spotlight (reference)



(f) Headlights off (key ’)

Figure 20: Gouraud Shading Mode vs **Phong Shading Mode** vs PBR, ceiling lights off to isolate the scanner spotlight (a, b, e) and the headlight point lights (c, d, f).

IMPLEMENTATION CARD — Phong / Gouraud shading **File:** `Shader.cpp`, `Main.cpp`
Function: `phongReflection()`, fragment `main()`, key `F5` in `handleDiscreteKey()`
Variables = current values: `uShadingMode` = 0 PBR / 1 Gouraud / 2 Phong; ambient 0.038; $n = \text{mix}(180, 10, r)$; k_s as above
If changed: raising the ambient 0.038 brightens unlit sides; larger n gives smaller, sharper highlights; larger k_s brighter highlights; switching 1↔2 shows per-vertex vs per-fragment.
Equation / evidence: Eq. (16); Fig. 20

5.3 Light types

Table 4: Light sources (all evaluated in `phongReflection()` and `pbrLighting()`).

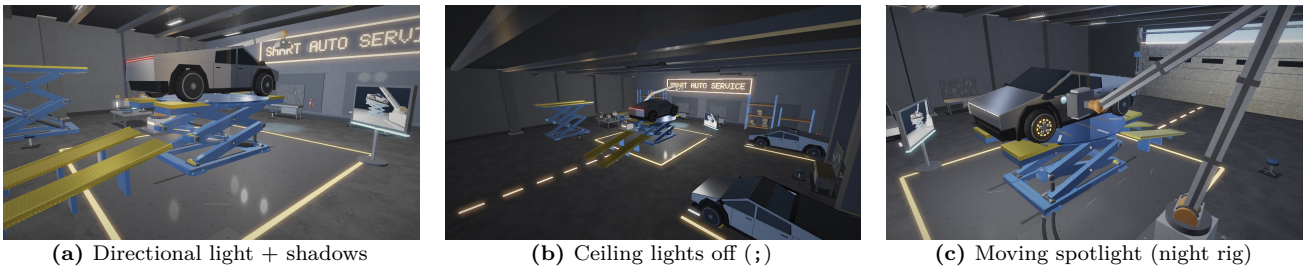
Light	Position / direction	Source	Control
Directional	$\hat{\mathbf{d}} = (0.18, \widehat{-1}, -0.38)$, colour $(0.95, 0.91, 0.84)$, $I = 0.68$; 5×5 PCF shadow	`buildLightSpaceMatrix()`, `buildWorkshopRig()`	`M` rigs, `F8` shadows
Point (ceiling, 8)	$(\pm 6.4, 6.32, z_i)$, $z_i \in \{-10.1, -3.9, 2.3, 8.5\}$ $(+0.55$ right row$)$, $I = 1.28$	`buildWorkshopRig()`	`;` $(1.28 \rightarrow 0.06)$
Point (headlights, 2)	$M_{\text{car}}(\pm 0.62, 0.923, -2.735)$, $I = 1.6$	`headlightLocalPosition()`, `kHeadlightIntensity`	’
Spotlight (moving)	at tool tip $\mathbf{t}$, direction $\widehat{\mathbf{t} - \mathbf{w}}$, cone 14°/23°, $I = 4.6$ cyan	`uploadSceneUniforms()`	robot keys, `F1`

Spotlight cone: $I_{\text{cone}} = \text{clamp}\left(\frac{\cos\theta - \cos 23^\circ}{\cos 14^\circ - \cos 23^\circ}, 0, 1\right)$ with $\cos\theta = -\mathbf{L}\cdot\hat{\mathbf{s}}$ (squared in PBR); at 18° off-axis the weight is 0.614. Because $\mathbf{t}, \mathbf{w}$ come from the joint angles each frame, it is a *moving light*.

IMPLEMENTATION CARD — Lighting rig **File:** `Main.cpp`, `SceneObjects.cpp`
Function: `buildWorkshopRig()`, `buildLightSpaceMatrix()`, `uploadSceneUniforms()`
Variables = current values: `bayLevel`=1.28, light $y = 6.32$; sun $(0.18, -1, -0.38)$, `keyIntensity`=0.68; spot cone 14°/23°, $I = 4.6$; `kHeadlightIntensity`=1.6; key `0`: night = moon 0.05
If changed: moving a z_i moves its light pool; raising `bayLevel` brightens the room; widening the outer cone enlarges the spot; changing the sun vector rotates every shadow.
Equation / evidence: Eq. (16); Table 4

5.4 Advanced extensions: PBR, shadows, textures, post-processing

Cook–Torrance PBR (default; `F5` cycles back to it): $f_r = \frac{k_d\, c}{\pi} + \frac{DFG}{4(\mathbf{n}\cdot\mathbf{v})(\mathbf{n}\cdot\mathbf{l})}$ with a GGX distribution ($\alpha = r^2$), Schlick Fresnel and Smith geometry [5, 6], anisotropic GGX on the stainless body, a clear-coat lobe and split-sum image-based lighting. **Shadows:** a 2048^2 directional map with 5×5 PCF (shadowed if $z - b > d_{\text{map}}$) and a 1024^2 spot map that follows the scanner [7]. **Textures:** a 4-layer `GL_TEXTURE_2D_ARRAY` (concrete, plaster, asphalt, tread plate) sampled by triplanar mapping, weights $\mathbf{w} = |\mathbf{n}|^6 / \|\cdot\|_1$, scale `textureScaleFor()` (concrete 0.34). **Post-processing:** RGB16F HDR with 4×MSAA, half-resolution bloom (6 passes) and ACES tone mapping [8].



(a) Directional light + shadows **(b)** Ceiling lights off (`;`) **(c)** Moving spotlight (night rig)

Figure 21: Light types and shadows (the same views with shadows off, `F8`, and textures off, `F6`, are in the program).

5.5 Dirt, cleaning, polishing and the changing reflection

The idea in one paragraph. The vehicle carries two numbers that the service changes over time: `dirtAmount` D ($1 \to 0$, grime on the left flank) and `polishAmount` p ($0 \to 1$, finish of the whole steel shell). The robot's *sponge* lowers D as it wipes; the robot's *polisher* raises p as it buffs. The fragment shader reads D and p every frame and changes how every pixel of the body is lit, so what you see changes with the tool. No texture is repainted: it is a change in the *material* (colour, roughness, metallic), and reflections follow automatically from the lighting equations. **1. Cleaning (the sponge).** In state CLEAN the sponge follows three bands of 4 waypoints each (high, middle, low; `PATH_CLEAN_HIGH/MID/LOW`). The dirt retreats *in step with the sponge's position*: $D = D_0 - \Delta \cdot \text{pathProgress}$, where pathProgress = waypoint$/(4-1)$. The three bands take D from $1 \to 0.65 \to 0.30 \to 0$ (code: `gServiceVisual.dirtAmount`), so the clean band follows the tool instead of fading on a timer. **2. How dirt is drawn (fragment shader).** A grime mask is built in the *vehicle's own frame* (so it stays on the panel when the car is lifted): it sits on the serviced flank only, collects low and thins upward (thrown up by the wheels), is heaviest behind the wheels, and is blotchy and streaked:

$$\text{mask} = \text{clamp}\Big(\underbrace{\text{side}\cdot\text{low}\cdot\text{along}}_{\text{where}}\cdot(0.35 + 0.95\,\text{blotch})(0.65 + 0.60\,\text{streak})\cdot D\cdot 2.25,\ 0,\ 1\Big), \tag{17}$$

$$\text{low} = 1 - \text{smoothstep}(0.48,\ 1.26,\ y_{\text{local}}).$$

Where the mask is m: albedo $\leftarrow$ mix(albedo, grime, $0.88\,m$) with grime $= (0.205, 0.183, 0.158)$; roughness $\leftarrow$ mix$(r, 0.94, m)$; metallic $\leftarrow$ metal $(1 - 0.92\,m)$; brushed grain and clear coat $\leftarrow (1 - m)$. The key point is the **metallic** term: dirt is a dielectric, so where the film is thick the surface stops behaving like a mirror — that is why grime *kills* a reflection instead of just tinting it. *Worked example* at height $y = 0.8$ (low = 0.633), blotch = streak = 0.5, $r = 0.40$, metal = 0.97:

D	mask m	roughness	metallic	grime weight $0.88m$
1.00 (dirty)	1.000	0.940	0.078	0.880
0.65 (after high band)	0.726	0.792	0.322	0.639
0.30 (after middle band)	0.335	0.581	0.671	0.295
0.00 (clean)	0	0.400	0.970	0

3. Polishing (the polisher). In state POLISH the buffer pad (Ø 0.58, spinning at 620°/s) makes six passes: high, middle and low band on the near flank, then the *turntable turns the car* 180° and the same three bands are polished on the far flank. p rises 0.17, 0.34, 0.50, 0.67, 0.84, 1.00 pass by pass, so the shine spreads as the pad moves. All steel batches of the shell take the same finish (`setSurfaceFinish(p)`), so a serviced car is polished

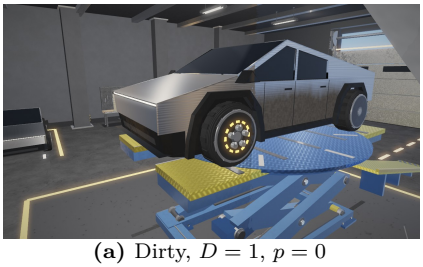
all over. **4. What polishing changes in the shader.** Three things, in this order:

$$r' = \mathrm{mix}(r,\ 0.34\,r+0.03,\ p) = r - p\,(r-0.166), \qquad \mathrm{aniso}' = \mathrm{aniso}\cdot(1-0.58\,p), \qquad \mathrm{albedo}' = \mathrm{albedo}\cdot(1+0.05\,p). \tag{18}$$

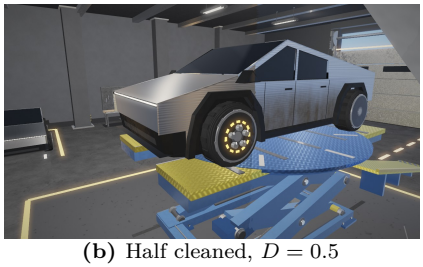
With $r = 0.40$ (as-rolled stainless) the roughness falls to 0.166 at $p = 1$, the brushed-grain directionality is cut to 42%, and the reflectance rises 5% (the dull oxide layer is cut away). No lacquer is added: a coat belongs to the material, not to how hard it was buffed. **5. Why the reflection changes.** Roughness enters the reflection in three places, and the table shows each for the three polish levels:

p	r'	GGX peak D_0	half-width	mip $6\sqrt{r'}$	Phong n
0 (as rolled)	0.400	12.4	5.99°	3.79	112
0.5	0.283	49.6	2.96°	3.19	132
1 (polished)	0.166	419	1.02°	2.44	152

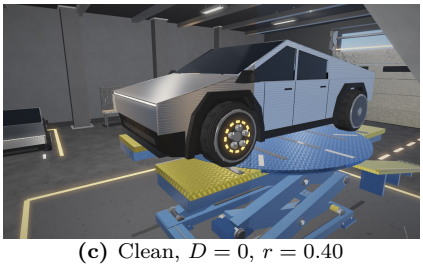
($D_0 = 1/(\pi r'^4)$ is the highlight peak, "half-width" the angle where D falls to $D_0/2$, mip $= 6\sqrt{r'}$ the cube-map level.) (a) *Light highlights.* The GGX distribution $D(\theta_h) = \alpha^2/\big(\pi[\cos^2\theta_h(\alpha^2-1)+1]^2\big)$, $\alpha = r'^2$, becomes 34 times higher and about six times narrower: a broad glow turns into a small hard glint of the ceiling lights (Fig. 22f). (b) *Environment reflection.* The room is a cube map with mip levels; the reflection ray reads level $6\sqrt{r'}$, so smaller r' reads a *sharper* mip and the reflected room becomes a clearer mirror image. (c) *Phong and Gouraud modes* use the shininess exponent $n = \mathrm{mix}(180, 10, r')$, which rises from 112 to 152: the same sharpening appears when you press `F5` — the effect is a material change, not a PBR-only trick. **How I demonstrate it.**



(a) Dirty, $D = 1$, $p = 0$



(b) Half cleaned, $D = 0.5$



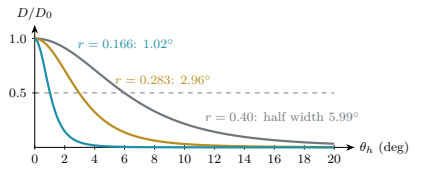
(c) Clean, $D = 0$, $r = 0.40$



(d) Half polished, $p = 0.5$, $r' = 0.283$

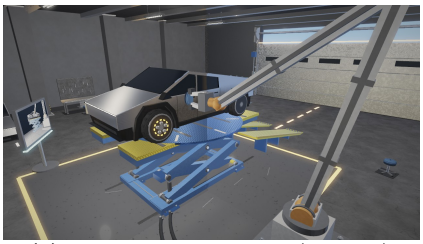


(e) Polished, $p = 1$, $r' = 0.166$

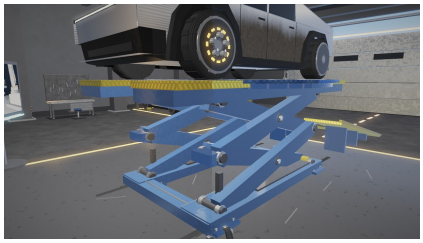


(f) Highlight shape D/D_0 narrows as p grows

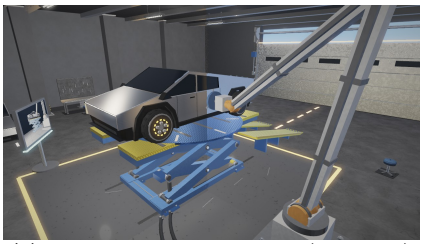
Figure 22: The same camera at five stages of the service (rendered with `-dirt` and `-polish`): grime disappears (a–c), then the steel gains sharp reflections (c–e); (f) is the GGX highlight of Eq. (18).



(a) Sponge wiping a band ($t = 86$ s)



(b) Grime still ahead of the sponge ($t = 84$ s)



(c) Polisher buffing the flank ($t = 110$ s)

Figure 23: The tools doing the work in the automatic sequence (stages CLEAN, 82.5–92.2 s, and POLISH, 103.0–136.0 s).

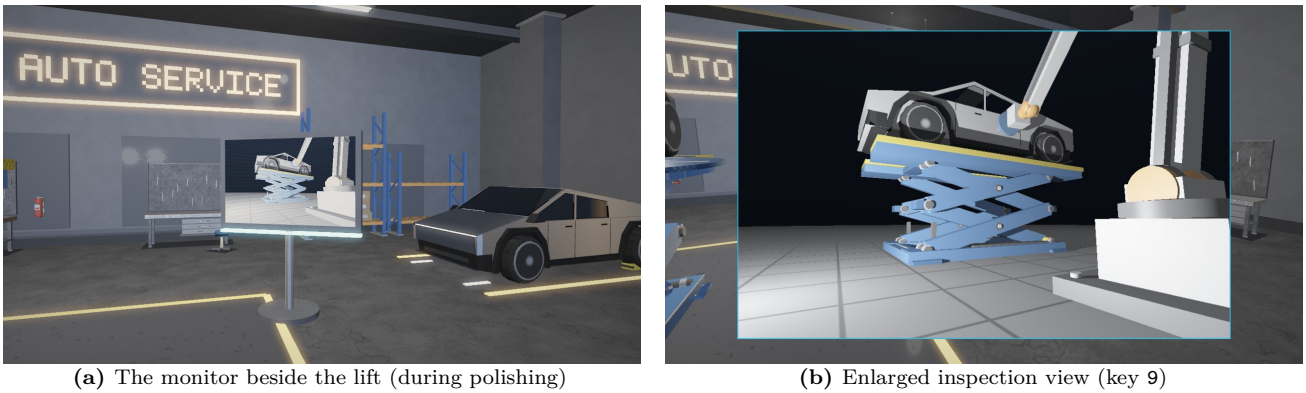
Press `Enter` and watch: from $t \approx 82$ s the sponge sweeps three bands and the grime on the left flank vanishes band by band, in step with the sponge; from $t \approx 103$ s the polisher buffs three bands, the turntable turns the car, and the shell goes from matte brushed steel to a sharp mirror finish, which is clearest in the glints of the ceiling lights. Press `F5` during the polish to compare PBR, Gouraud and Phong on the same surface, and `F9` to switch the environment reflection off and on. The captures in Fig. 22 use the program's `-dirt D` and `-polish p` switches to freeze D and p at the values shown.

IMPLEMENTATION CARD — Dirt, cleaning, polishing, reflection **File:** `Main.cpp`, `SceneObjects.cpp`, `Shader.cpp`
Function: `updateServiceAutomation()` (CLEAN, POLISH), `drawCar()`, `Scene::setSurfaceDirt()`, `Scene::setSurfaceFinish()`, fragment `main()` (finish and grime resolve), `environmentColor()`
Variables = current values: `dirtAmount` $1 \rightarrow 0$ in 3 bands (0.65, 0.30, 0); `polishAmount` $0 \rightarrow 1$ in 6 passes (+0.17 each); `kStainlessAsRolled`=0.40; polish: $r' = 0.34r + 0.03$, aniso $\times 0.42$, albedo $\times 1.05$; grime colour $(0.205, 0.183, 0.158)$; rough 0.94; metallic $\times 0.08$
If changed: a larger grime factor (2.25) makes the dirt cover more; a lower `kStainlessAsRolled` starts with a shinier panel (less visible change); changing $0.34r + 0.03$ sets how mirror-like the final polish is; raising the mirror mip scale (6) blurs all reflections; `F9` removes the environment term.
Equation / evidence: Eqs. (17), (18); Figs. 22, 23

6 Ray-Traced Underbody Inspection Monitor

6.1 Purpose and an honest description

The garage is a **rasterised** smart garage: the main image is drawn with the OpenGL pipeline described in the previous sections. In addition, a physical **inspection monitor** stands beside the hydraulic lift (Fig. 24), and the image on that monitor is produced by **genuine ray tracing** in a separate GLSL pass. Its garage purpose is practical: the operator can watch the *underside* of the raised vehicle, its wheels, the lift supports and the robot tool from a low viewpoint that the normal garage camera cannot take — and, because every pixel is traced, the lift arms and the robot cast real shadows on the car and the floor. The monitor does *not* detect faults; it only shows the geometry.



(a) The monitor beside the lift (during polishing) **(b)** Enlarged inspection view (key `9`)

Figure 24: The ray-traced monitor in the garage (left) and enlarged for demonstration (right).

6.2 Pipeline: one extra pass per frame

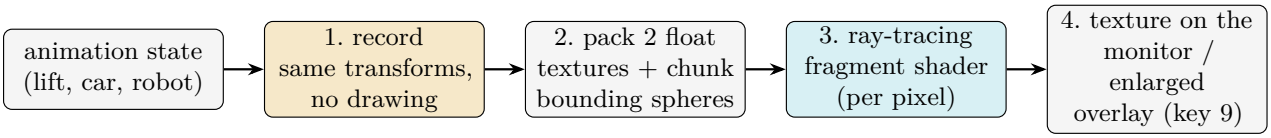


Figure 25: The four steps of the monitor pass (`rtRender()` in `Main.cpp`); the main garage renderer is unchanged.

1. **Record.** `Scene::recordInspectionGeometry()` (SceneObjects.cpp) re-runs `drawHydraulicLift()`, `drawCar()` and `drawRobotArm()` in *record mode*: `drawCube`, `drawCylinder`, `drawSphere` and the car-shell batches append what they would have drawn — with the *same matrices* — to a list instead of drawing. The lift, the car (via `serviceCarMatrix()`), the wheels (`wheelSpin`) and the robot (all five joints and the fitted tool) therefore follow the live animation state with no second copy of the logic.
2. **Pack.** The list (790–793 items) is written into an RGBA32F texture, four texels per item, and sorted along a Morton (Z-order) curve of their centres and grouped into chunks of 16 consecutive items, each with a bounding sphere.
3. **Trace.** A fullscreen fragment shader (`kRayTraceFragmentShader`, `Shader.cpp`) runs once per monitor pixel into an off-screen RGBA8 texture (512×320, refreshed every second frame; 1024×640 every frame when enlarged).
4. **Display.** The texture is mapped on the monitor's screen quad inside the garage (`rtDrawScreen()`) and, on key `9`, drawn as a large overlay on top of the finished frame (`rtDrawOverlay()`). The pass restores the framebuffer, viewport, program and enable-state it changes.

6.3 Ray generation

For pixel coordinates $(u, v) \in [-1, 1]^2$, aspect $a = 512/320 = 1.6$ and vertical field of view $\varphi = 64°$ ($\tan\frac{\varphi}{2} = 0.625$), the camera ray is

$$\mathbf{o} = \mathbf{c}, \qquad \mathbf{d} = \text{normalize}\Big(\mathbf{f} + u\,a\tan\tfrac{\varphi}{2}\,\mathbf{r} + v\tan\tfrac{\varphi}{2}\,\mathbf{u}\Big), \tag{19}$$

where $\mathbf{c}$ is the camera position and $(\mathbf{f}, \mathbf{r}, \mathbf{u})$ its forward, right and up vectors (`rtCamera()`). The camera looks at a point that rises with the lift, $\mathbf{T} = (0,\ 0.5\,y_{\text{top}} + 0.45,\ -2)$ with $y_{\text{top}} =$ `liftTopY`(h) (for the robot-side preset $\mathbf{T} = (-0.72,\ 0.5\,y_{\text{top}} + 0.52,\ -2.10)$), so raising the car changes the image. Four presets (key `=`, `gRtPreset`, names in `kRtPresetNames`) place the camera low beside or in front of the car: **0** rear underbody $\mathbf{c} = (0.35, 0.55, 6.8)$; **1** nose underbody $(0.35, 0.55, -9.0)$; **2** opposite-side overview $(5.4, 0.75, -3.6)$; **3** *live robot-side diagnostic* $(-5.75, 0.78, -5.10)$. Preset 3 is the **default**: it watches the same $-x$ flank that the robot services, so the arm, its tool, its shadow and the panel being worked on are all in view while the service runs. `Insert`/`Delete` orbit the camera about $\mathbf{T}$ (`gRtYaw`); pressing `=` resets the orbit.

6.4 Intersections

Each item is tested against the ray $\mathbf{o} + t\mathbf{d}$, $|\mathbf{d}| = 1$. Boxes, cylinders and ellipsoids are the *unit shapes* of the rasteriser (cube $[-\frac{1}{2}, \frac{1}{2}]^3$, cylinder $r = \frac{1}{2}$, $y \in [-\frac{1}{2}, \frac{1}{2}]$, sphere $r = \frac{1}{2}$) placed by their model matrix M. The shader stores M^{-1} and transforms the ray into the shape's own space, $\mathbf{o}' = M^{-1}\mathbf{o}$, $\mathbf{d}' = M^{-1}\mathbf{d}$ (not renormalised, so the same t is valid in both spaces):

$$\text{box:}\quad t_{1,2} = \frac{\mp\frac{1}{2} - o'}{d'}\ \text{(per axis)},\quad t_n = \max\min(t_1, t_2),\ t_f = \min\max(t_1, t_2),\ \text{hit iff } t_n \le t_f,\ t_f > 0; \tag{20}$$

$$\text{cylinder:}\quad a = d_x'^2 + d_z'^2,\ b = o_x'd_x' + o_z'd_z',\ c = o_x'^2 + o_z'^2 - \tfrac{1}{4},\quad t = \frac{-b \mp \sqrt{b^2 - ac}}{a},\ |o_y' + td_y'| \le \tfrac{1}{2},\ \text{plus caps } y = \pm\tfrac{1}{2}; \tag{21}$$

$$\text{sphere:}\quad |\mathbf{o}' + t\mathbf{d}'|^2 = \tfrac{1}{4}; \qquad \text{triangle (Möller–Trumbore):}\quad t = \frac{\mathbf{e}_2 \cdot (\mathbf{s} \times \mathbf{e}_1)}{\mathbf{e}_1 \cdot (\mathbf{d} \times \mathbf{e}_2)},\ \mathbf{s} = \mathbf{o} - \mathbf{a}. \tag{22}$$

The world normal of a box, cylinder or sphere hit is $\mathbf{n} = \text{normalize}\big((M^{-1})^T\mathbf{n}'\big)$; a triangle's is the normalised $\mathbf{e}_1 \times \mathbf{e}_2$ (flipped to face the ray). The floor is the analytic plane $y = 0$, $t = -c_y/d_y$.

6.5 Nearest hit, shading and the shadow ray

Nearest surface. The tracer keeps the smallest valid t found so far (`tBest`); every later candidate must beat it (`hitItem(...,tBest,...)`), so a farther surface can never replace a nearer one. The object hit is compared with the floor distance the same way. Before looking inside a chunk, the ray is tested against the chunk's bounding sphere; a chunk that the ray misses, or whose nearest possible point lies beyond `tBest`, is skipped.

Shading and shadow ray. At the hit point P with normal $\mathbf{n}$ (flipped to face the viewer) and material colour $\mathbf{k}$ (the colour of the primitive the rasteriser uses), the inspection light at P_L — one of five presets in `kRtLights` (key `PageDown`); the default (index 2) is a floor lamp on the robot side at (−2.6, 0.6, −6.0), and the numpad moves it freely — gives

$$I = \mathbf{k}\Big(0.085 + 0.06\,n_y + 0.05\max(\mathbf{nV}, 0)\Big) + A\,v\Big[\mathbf{k}\max(\mathbf{nL}, 0) + 0.30\max(\mathbf{nH}, 0)^{48}\Big], \qquad A = \frac{4.6}{1 + 0.10\,d + 0.04\,d^2},\ d = |P_L - P|, \tag{23}$$

and an exposure curve $1 - e^{-1.35I}$ with gamma 1/2.2. The visibility v comes from a **shadow ray**: origin $P + 0.012\,\mathbf{n}$ (to avoid hitting its own surface), direction $\mathbf{L} = (P_L - P)/d$ and

$$t_{\max} = d - 0.03, \qquad v = \begin{cases} 0 & \text{if any item has a hit with } 0 < t < t_{\max} \\ 1 & \text{otherwise,} \end{cases} \tag{24}$$

so only blockers *between* the surface and the light count; anything behind the light is ignored (Fig. 26). When shadow rays are switched off (key `-`) the same ray, camera, geometry, colours and light give $v = 1$, which is the comparison of Fig. 28.

6.6 What is traced, acceleration structure and approximations

Acceleration structure. A one-level bounding-volume hierarchy: consecutive items (which are spatially close, because they are drawn part by part) form chunks of 16 (after Morton sorting) with a bounding sphere computed

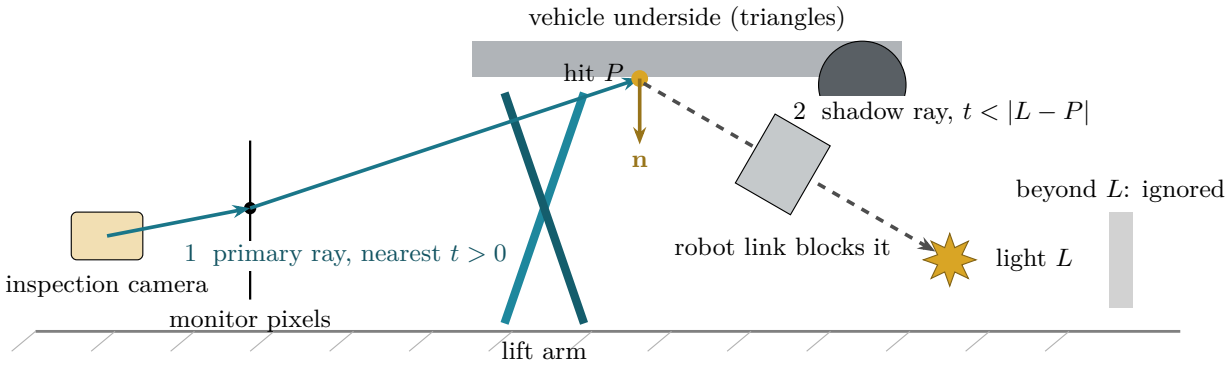


Figure 26: Ray path: (1) the camera ray through a monitor pixel finds the nearest hit P on the underside; (2) a shadow ray runs from P toward the light L; a robot link between them blocks it (shadow), a blocker beyond L is ignored.

Table 5: Contents of the traced scene at the default state (lift raised, robot at the tool station; counts printed by the program with `-capture`).

Object	Traced as	Items
Hydraulic lift (base, scissor arms, pins, rams, runways)	boxes, cylinders (the lift's own primitives)	114
Serviced vehicle: underbody, side skin, arches, bumpers, trim	triangles of the car-shell batches (glass and lamps omitted)	452
Vehicle wheels and detail parts	cylinders and boxes of `drawWheel()` and `drawCar()`	184
Robot arm and the fitted tool	boxes, cylinders, spheres of `drawRobotArm()`	41–43
Floor	analytic plane $y = 0$	–
Total		**793** (50 chunks of 16)

on the CPU every frame. A ray visits a chunk's items only if it enters the sphere. With ≈ 790 items this gives an interactive monitor on an OpenGL 3.3 GPU; a deeper BVH would be needed for much larger scenes. **Measured cost** (integrated Intel Iris Xe, capture frame with `glFinish`): about 25 ms for the 512×320 monitor image and about 45 ms for the enlarged 1024×640 view, which is why the small monitor refreshes every second frame; a dedicated GPU is faster.

Documented approximations. The monitor is *not* an exact copy of the visible garage: (i) the car is its shell triangles without glass or lamps; (ii) details smaller than 6 cm (bolts, rivets) are dropped; (iii) the turntable plate, hoses, tool station and the rest of the building are not traced, so the underside of the car is visible; (iv) the floor is a plane with painted grid lines; (v) there is one light, one shadow ray and no reflections or global illumination. The vehicle components shown are exactly those the project models — none are invented.

6.7 Controls

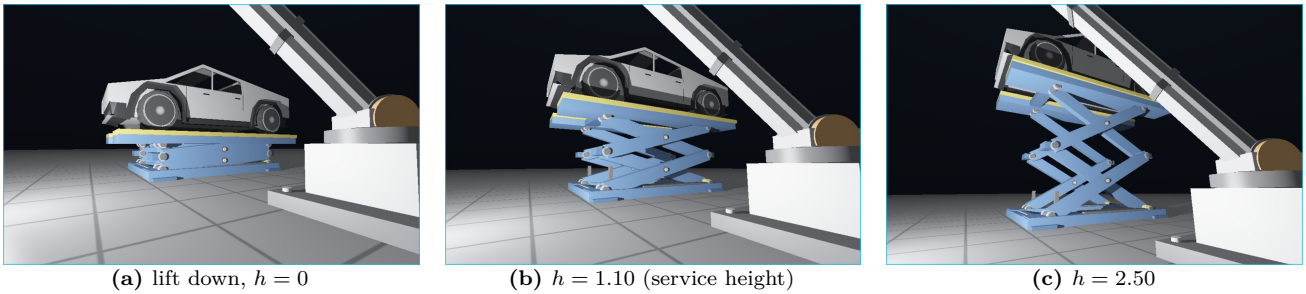
Table 6: Inspection-monitor controls (keys checked against the existing bindings; all are new).

Key	Action	Variable
`9`	enlarge / close the inspection view (re-renders at 1024×640)	`gRtLarge`
`-`	inspection-light shadow rays on / off	`gRtShadows`
`=`	next camera preset (rear, nose, opposite side, robot side = default)	`gRtPreset`
`Insert` / `Delete`	orbit the inspection camera	`gRtYaw`
`PageDown`	next light position (5 presets; default: robot-side floor lamp)	`gRtLightIdx`
Numpad `4 6 8 2 + -`	move the inspection light freely (x, z, height)	`gRtLight`

Command-line switches for repeatable figures: `-rtfull`, `-rtlarge`, `-rtpreset`, `-rtshadow`, `-rtlight x y z`, `-rtlightidx`, `-rtyaw`. The existing keys, shading modes, lighting, car, lift, robot and the 18-state service are unchanged.

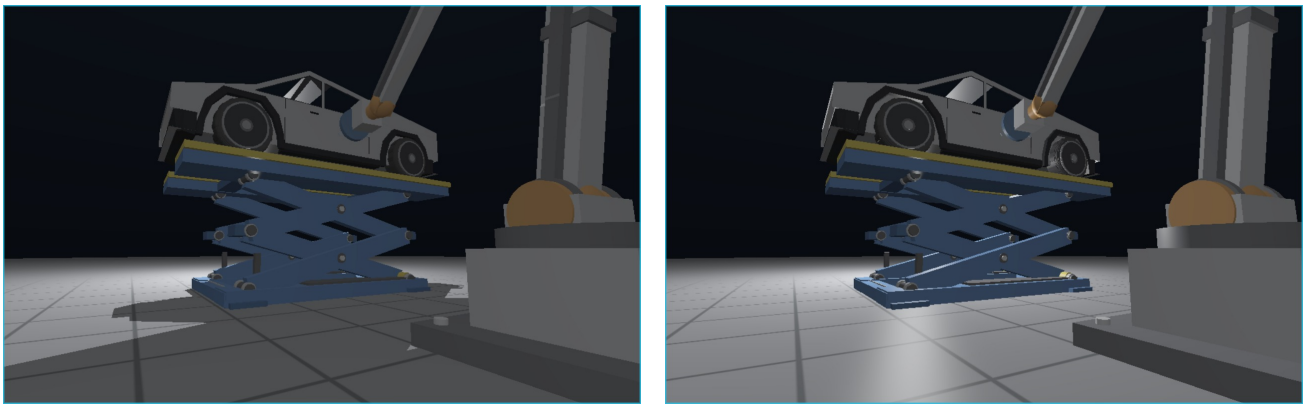
6.8 Results and verification

IMPLEMENTATION CARD — Ray-traced inspection monitor **File:** `Shader.cpp`, `Main.cpp`, `SceneObjects.cpp`
Function: `kRayTraceFragmentShader` (`hitItem`, `traceClosest`, `occluded`), `rtRender()`, `rtCamera()`, `Scene::recordInspectionGeometry()`, `Scene::inspectionMonitorMatrix()`
Variables = current values: $\varphi = 64°$; 512×320 / 1024×640; default preset 3 (robot side), light index 2; chunk size 16; light power 4.6; shadow bias 0.012; $t_{\max} = d - 0.03$; 793 items; monitor at $(4.4, 0, -5.6)$, screen 2.0×1.25 m
If changed: a larger chunk size (16 now) tests fewer spheres but more items per hit; raising the light power brightens the picture; a larger bias removes self-shadowing at grazing angles but detaches shadows; more items (a less aggressive size cut-off, 6 cm) slows the pass; `-` removes the shadow term.
Equation / evidence: Eqs. (19)–(24); Figs. 25–29



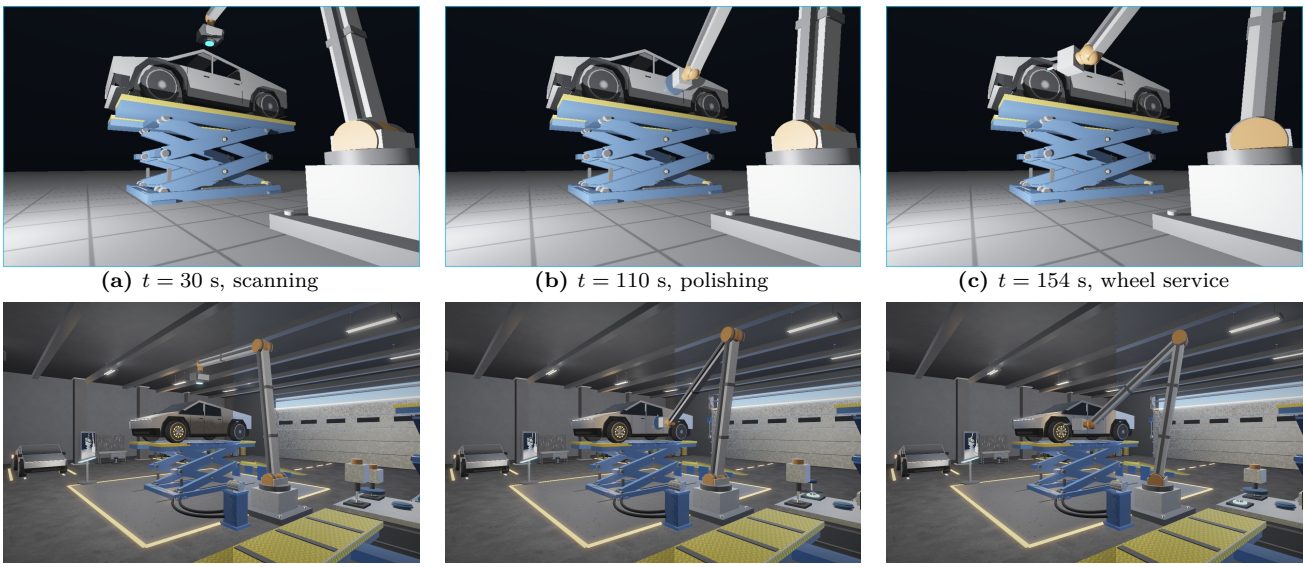
(a) lift down, $h = 0$ **(b)** $h = 1.10$ (service height) **(c)** $h = 2.50$

Figure 27: Raising the car changes the traced image: the scissor stack opens, the car rises and the camera follows the raised target.



(a) shadow rays on **(b)** shadow rays off

Figure 28: Same robot-side camera, geometry, colours and light (light preset 3 at $(3.4, 2.6, -2)$, state at $t = 110$ s, polishing): only v of Eq. (24) changes. With shadows on, the scissor lift and the car cast their shadow across the floor.



(a) $t = 30$ s, scanning **(b)** $t = 110$ s, polishing **(c)** $t = 154$ s, wheel service

(d) garage view, $t = 30$ s **(e)** garage view, $t = 110$ s **(f)** garage view, $t = 154$ s

Figure 29: Synchronisation: the traced monitor in its default robot-side view (top) and the garage seen from the robot side (bottom) at the same simulation times; the robot arm, its tool and the turntable-rotated car match.

Table 7: Verification performed with the final build.

Check	How	Result
Raising the car changes the image	captures at $h = 0$, 1.10, 2.50 (Fig. 27)	scissor opens, car rises: passed
Wheels and robot synchronised	monitor and garage captured at $t = 30, 110, 154$ s (Fig. 29)	arm, tool, car yaw match: passed
Nearest surface hides what is behind	wheels and arches hide the lift behind them in the robot-side view; `tBest` logic	passed (visual and code)
Shadow rays only use blockers before the light	$t_{\max} = d - 0.03$; blockers beyond the light have no effect	passed (code; Fig. 26)
Same scene with shadows on/off	identical capture arguments except `-rtshadow` (Fig. 28)	only the shadows differ: passed
Enlarge / close	`-rtlarge`, overlay at 1024×640 (Fig. 24b)	passed
Existing service still runs	`-simtime` to 154 s and the full 197.7 s sequence reach COMPLETE	passed
Builds	x64 Release and Debug	no errors, no compiler warnings
Not performed	live interactive key presses in a window, other GPUs, shader-failure fault injection (the failure path returns before any monitor state is used)	stated honestly

7 Motion, Animation, Interaction and Collision

Interpolation. Door, lift and drive use constant-speed `moveToward()`: $p \leftarrow p + \text{clamp}(p_1 - p, \pm v\Delta t)$. Robot moves share one eased timeline (`moveRobotTowardPose()`):

$$u = \min\big(t/T, 1\big),\ s(u) = 3u^2 - 2u^3,\ \theta_j(t) = \theta_j^0 + s(u)\,\text{wrap}_{180}(\theta_j^1 - \theta_j^0),\ T = \max\Big(0.22, \tfrac{\max_j |\Delta_j|}{52k}\Big), \tag{25}$$

i.e. $p(t) = (1-s)p_0 + sp_1$ with zero speed at both ends; wrapping takes the short way ($170° \to -170°$ is $+20°$).

Vehicle and wheels. `carDrive` changes at 0.092/s, so the 18.5 m run takes 10.87 s (1.70 m/s). The ramp is sampled under each axle (Eq. (10), Fig. 13). Rolling without slipping:

$$\Delta\phi_{\text{deg}} = \frac{\Delta s}{r}\frac{180}{\pi} \ \Rightarrow\ \phi_{\text{run}} = \frac{18.5}{0.450}\frac{180}{\pi} = 2355.5° = 6.54 \text{ turns}. \tag{26}$$

Everything that moves (variable, rate): vehicle `carDrive` 0.092/s (Eq. (14)); wheels `wheelSpin` (Eq. (26)); door `gDoorOffset` 0.8 m/s; lift `gLiftHeight` 0.55 m/s; robot joints Eq. (25) at 52°/s; turntable `carYaw` 34°/s the short way; polisher pad 620°/s and impact-driver socket 980°/s; gripper opening 1.8–2.4/s and lug nuts 1.35/s (they seat 30 mm); warning beacon $0.35 + 0.65(\frac{1}{2} + \frac{1}{2}\sin 9t)$ while the door moves.

Interaction. Discrete keys are edge-detected once per press (e.g. `F5`: `gShadingMode = (gShadingMode + 1) % 3`); held keys change a value every frame and are validated by `robotPoseSafe()` before being applied (Table 8).

Table 8: Controls grouped by feature (verified against `handleDiscreteKey()` and `processMovement()` in `Main.cpp`).

Input	Action	Variable / function	Visible effect
CAMERA			
`W A S D`, `Space/Ctrl` (`E/Q`), `Shift`	move (7 m/s, ×2.5 with Shift)	`gCamera.position`	fly through the garage
Mouse, scroll, `Tab`	look, zoom (FOV 20–90°), free the cursor	`yaw, pitch, fov`	view direction and zoom
DOOR, LIFT AND VEHICLE			
`0`; `Z/X/C`	door auto toggle; close / open / stop	`gDoorTarget`, `gDoorOffset`	roller door moves
`P`; `[ ] \`	lift auto (3.20 / down); lower / raise / stop	`gLiftTarget`, `gLiftHeight`	lift moves (collision-checked)
`Home`; `PgUp`	vehicle out / in; back to the apron	`gCarDriveTarget`	vehicle drives
ROBOT			
`T/G`, `U/J`, `I/K`, `Y/H`, ← / →	base yaw, shoulder, elbow, wrist pitch, wrist roll	Fig. 9	arm and spotlight move
`F1`–`F4`	scanner / driver / gripper / polisher	`gRobotTool`	tool head changes
LIGHTING			
`;`, `'`, `/`, `End`	ceiling lights, headlights, neon, beacon	`gSwitches.*`	lights switch
`0`	outside light: daylight ↔ night	`gNight`, `applyDayNight()`	blue sky and sun / dark sky and dim moon
`M`; `,` `.`	lighting preset; reflection (IBL) intensity	`gLightingPreset`, `gIblIntensity`	whole rig / reflections change
INSPECTION MONITOR (ray tracing)			
`9`, `-`, `=`	enlarge/close; shadow rays on/off; camera preset	`gRtLarge`, `gRtShadows`, `gRtPreset`	monitor image changes (Section 6)
`Insert/Delete`, `PageDown`, numpad	orbit camera; light preset; move light	`gRtYaw`, `gRtLightIdx`, `gRtLight`	view and shadows change
SHADING AND PROJECTION			
`F5`	PBR → Gouraud → Phong	`gShadingMode`	shading model changes
`F6`–`F11`	textures, bump, shadows, reflections, bloom, MSAA	`gTexturesEnabled` ...	feature on / off
`Backspace`	perspective / orthographic	`gOrthographic`	projection changes
AUTOMATIC SERVICE, DEBUG AND TEACHING			
`Enter`, `R`	start / reset the automatic service	`startServiceDemo()`	full sequence (Section 8)
`L`, `1`–`8`, `N/B`, `V`, `F`, `F12`	wireframe, isolate object, build stage, report, screenshot, orbit	various	teaching and demo aids

Collision. The arm is three swept segments (shoulder→elbow, elbow→wrist, wrist→tool tip; radii 0.20, 0.18, 0.22–0.28 m) tested against AABBs inflated by the radius (slab test, `segmentHitsAABB()`: per axis $t_{1,2} = (b_{\min,\max} - a)/d$, hit iff $\max_{\text{axes}} \min(t_1, t_2) \le \min_{\text{axes}} \max(t_1, t_2)$ within the segment); boxes overlap iff $A_{\min} \le B_{\max} \wedge A_{\max} \ge B_{\min}$ on every axis [9]. Checked: pedestal, tool table, other lifts, power pack, ramp, ceiling fixtures, main lift, turntable disc and the vehicle (AABB of the rotated footprint, $h_x = |\cos\psi|1.34 + |\sin\psi|2.95$). In service mode the tool may approach the paint but never enter the solid shell/wheel boxes; a blocked manual move is rejected and the title shows "SAFETY STOP". *Example* (`POSE_LUG_1`, $h = 1.10$): the driver tip is at $x = -1.305$ and the inflated wheel box starts at $x = -1.25$, giving 55 mm clearance, so contact is allowed.

IMPLEMENTATION CARD — Collision safety **File:** `Main.cpp`
Function: `robotPoseSafe()`, `segmentHitsAABB()`, `segmentSegmentDistance()`, `liftPositionSafe()`
Variables = current values: segment radii 0.20/0.18/0.28; vehicle solid $x \in [-1.03, 1.03]$, $y \in [y_t{+}0.28,\ y_t{+}1.80]$; wheels $|x| \le 1.24$; margin 0.010; workspace $0.18 \le y \le 6.72$
If changed: larger radii make the arm stop earlier (more conservative); shrinking a box lets the tool approach closer; setting `allowVehicleServiceContact`=false keeps the tool out of the whole approach volume.
Equation / evidence: slab test; overlap condition above

8 Automatic Service Workflow

`updateServiceAutomation(dt)` switches on `gServiceState`; each state advances `gServicePhase` when its completion condition holds (Fig. 30; times measured with the fixed-step `-simtime` capture mode). Every state has the same shape: set targets, wait for the completion condition, call `setServiceState()`. **Completion conditions**

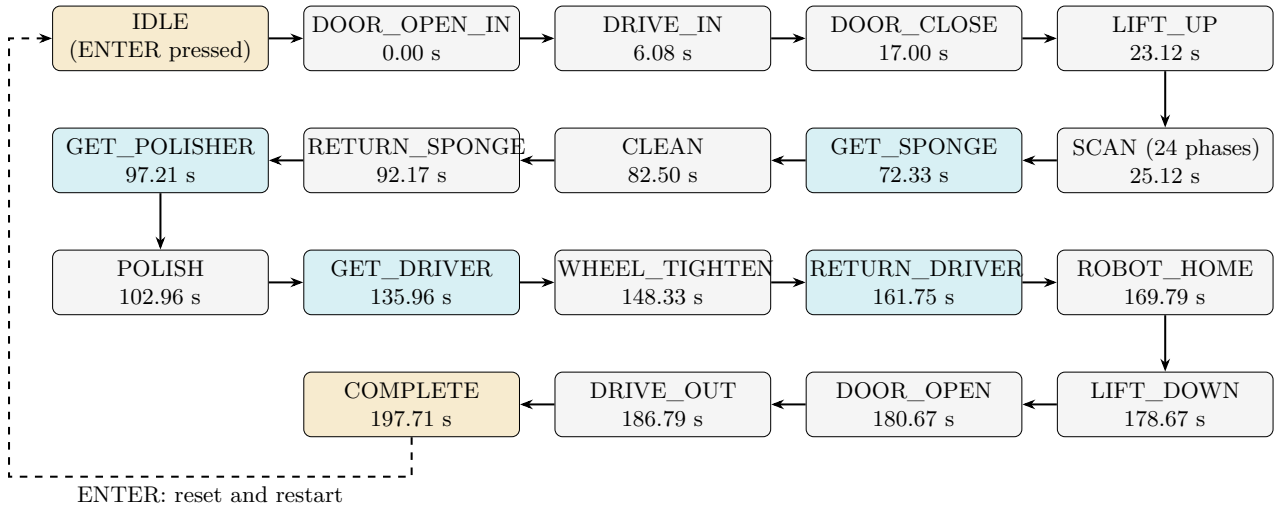


Figure 30: Automatic service state machine (18 states, 197.7 s); blue states perform the visible 10-phase tool exchange (`runToolChange()`).

(all in `Main.cpp`). DOOR_OPEN_IN: `gDoorOffset`≥ 4.80; DRIVE_IN: `carDrive`≤ 0.002; DOOR_CLOSE: door ≤ 0.035; LIFT_UP: $|h - 1.10| < 0.025$; SCAN: 24 pose phases (6 regions, turntable 180° and back); GET_SPONGE / GET_POLISHER / GET_DRIVER: the 10-phase tool exchange; CLEAN: `dirtAmount` $1 \rightarrow 0$ in 3 wiping bands; POLISH: `polishAmount` $0 \rightarrow 1$ (16 phases, both flanks); WHEEL_TIGHTEN: star order 1-3-5-2-4 with `lugProgress[i]`; ROBOT_HOME: HOME pose reached; LIFT_DOWN: $h \leq 0.025$; DOOR_OPEN; DRIVE_OUT: `carDrive`$\geq 0.998 \rightarrow$ COMPLETE.

9 Requirement Fulfilment and Final Verification

FINAL VERIFICATION. Proposal completion: 21 / 21 (Table 1) | **Mandatory requirements:** Lighting ✓ Transformation ✓ Gouraud ✓ Phong ✓ Motion ✓ Interaction ✓ | **Build:** x64 Debug ✓ Release ✓ (no errors; the only messages are linker notices that `glfw3.lib` ships without debug symbols). Values in this report were read from the source constants; every scene figure was rendered by the final build. The inspection monitor is ray traced (Section 6); the rest of the garage is rasterised, and no ray tracing is claimed for it.

10 Conclusion

All 21 proposal items and all six mandatory requirements are implemented and can be shown live; PBR, shadows, textures, HDR, the tool-exchange service and the ray-traced inspection monitor (bonus) go beyond the proposal.

Technical references and development sources. The theory follows the cited OpenGL 3.3 and GLSL specifications, *Learn OpenGL*, Phong (1975), Gouraud (1971), Blinn (1977), Cook–Torrance (1982), GGX/-Smith/Schlick, the UE4 and Disney shading courses, the ACES fit and Ericson's collision text. The only third-party code is GLFW (window and input) and GLAD (OpenGL loader), bundled in `Libraries/` and `glad.c`; the project loads no model or image files — meshes, textures and shaders are generated by the six project source files (`Math3D.h`, `Camera.h`, `Primitives.cpp`, `SceneObjects.cpp`, `Shader.cpp`, `Main.cpp`).

References

[1] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification (Version 3.3 (Core Profile))*. The Khronos Group Inc., 2010.

[2] J. de Vries, *Learn OpenGL: Learn Modern OpenGL Graphics Programming in a Step-by-Step Fashion*. Kendall & Welling, 2020.

[3] B. T. Phong, "Illumination for computer generated pictures," *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.

[4] H. Gouraud, "Continuous shading of curved surfaces," *IEEE Transactions on Computers*, vol. C-20, no. 6, pp. 623–629, 1971.

[5] R. L. Cook and K. E. Torrance, "A reflectance model for computer graphics," *ACM Transactions on Graphics*, vol. 1, no. 1, pp. 7–24, 1982.

[6] B. Karis, "Real shading in Unreal Engine 4," in *SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice*, 2013.

[7] L. Williams, "Casting curved shadows on curved surfaces," in *Proceedings of SIGGRAPH '78*, pp. 270–274, 1978.

[8] K. Narkowicz, "ACES filmic tone mapping curve." Blog post, knarkowicz.wordpress.com, 2016.

[9] C. Ericson, *Real-Time Collision Detection*. Morgan Kaufmann, 2005.